

概述

Hello, React :-)

React 是用于构建用户界面的 JavaScript 库，React 核心只关注视图，不断优化算法，改进性能，提高开发和交互体验。

React 迭代稳定，重视兼容和过渡，在国内外，尤其是南方，都有相当多的公司在使用 React。

渐进式的思想同样表现在 React 的学习曲线上，能够与传统的 Web 技术共存，灵活的 JSX 语法等都会让 React 上手很快，而庞大生态赋予了 React 更强能力的同时，也让开发者感叹花费了更多时间在社区里遨游。

React 应用

React 在前端开发领域应用广泛，使用 React 可以构建 Web，插件，单页应用，App，小程序，桌面端，服务端等，微服务，Serverless，低代码，虚拟现实等都有 React 的用武之地。

React 面试注意事项

React 面试题可以分为以下 4 个方面

- 基础：ES5+ 作用域，class，箭头函数，this 指向，异步编程，高阶函数的循环等常问
- 会用：state，副作用，Hook，加载渲染过程，路由、测试、调试、TS、Redux 等常问
- 原理：Virtual DOM，Diff 算法，设计组件，优化性能，原理和实现等高级岗位、大厂常问
- 项目：项目结构，技术栈，工具链，解决问题，担任角色，亮点等常问，也是面试问题来源

新手最好先熟练 ES5+ 的使用，再上手 React 时，可以边阅读边写代码，适当练习若干项目，再看原来生涩的表述，会有亲切的画面感。带着项目去看面试题，联想练习或工作中遇到的实际问题，加上自己的理解，你的回答一定可以比手册总结得更自然，更能得到面试官的肯定。

认识

对比 React 和 Vue

相同点

- 支持 Virtual DOM
- 支持响应式和组件化的视图组件
- 核心库、路由和状态管理分离
- 支持 JSX，移动端都支持原生渲染

不同点

- 预编译
 - React 可以通过 Prepack 优化 JavaScript 源代码，在编译时执行原本在运行时的计算过程，通过简单的赋值序列提高 JavaScript 代码的执行效率，消除中间计算过程及分配对象操作。缓存 JavaScript 解析结果，优化效果最佳
 - Vue 可以静态分析 template，构造 AST 树，通过 PatchFlags 标记节点变化类型
- 渲染
 - React 通过 shouldComponentUpdate / setState，使用 PureComponent 等对比前后状态和属性，手动决定是否渲染来优化
 - Vue 推荐模板语法，自动追踪组件依赖，精确渲染状态改变的组件
- 事件处理
 - React
 - React 17前，事件委托到 document，之后委托到 根节点
 - 所有事件被合并为合成事件并兼容不同浏览器
 - 事件处理函数中的 this 需要手动绑定或使用箭头函数声明
 - Vue
 - 原生事件
 - this 自动绑定执行上下文

什么是 React

React 是用于构建用户界面的 JavaScript 库

- 声明式编写 UI，代码可靠，便于调试
- 组件化开发，组件逻辑使用 JavaScript 编写而非模板，遵循单向数据流和数据绑定，状态与 DOM 分离
- 一次学习，随处编写，使用 Virtual DOM，支持 浏览器、Node 服务器等多种渲染方式和 React Native 开发原生应用

对比 React 和 Angular

- 核心功能
 - React 核心库只提供构建 UI 组件的方法，其他功能通过社区提供
 - Angular 集成了 路由、异步请求、表单、模块化 CSS 等功能
- 组件

- React 组件推荐使用 JSX，可以一个文件包含 HTML、CSS 和 JS，也可以分开
- Angular 组件 HTML、CSS 和 TS 分别是一个文件
- DOM
 - React 基于 Virtual DOM，组件会被编译成 JS 对象，数据更改时通过 Diff 算法更新
 - Angular 基于 Incremental DOM，组件会被编译成指令，数据更改时就地更新。没有使用规定指令的组件可以被 Tree Shaking
- 数据绑定
 - React 单向数据绑定，声明状态，更新视图
 - Angular 双向数据绑定，数据改变，更新视图
- 全局状态管理
 - Angular 可以用 Service 依赖注入实现
 - React 可以用全局对象或 Redux 实现
- 上手成本
 - React 推荐了解 JSX，是库，可以渐进式使用
 - Angular 需要了解 TypeScript，Rxjs，OOP 和装饰器等，是框架，推荐独立使用

你认为 React 的缺点是什么

- React 核心是 UI 库，路由，状态管理等由社区维护。细粒度需求和问题依赖社区解决
- React 概念和约束较少，容易上手并与现有项目整合。代码风格和项目结构容易产生差别
- React JSX 灵活性高，预编译时可以做的优化相对其他 HTML 分离的库有限
- React setState 提供基于队列异步更新，手动优化渲染流程，需要关注业务之外的逻辑
- React17 以前基于事件委托的合成事件，表现和使用与原生事件存在差异

什么是声明式编程

- 声明式编程是一种编程范式，描述目标，而不是流程
- 通过函数、推理规则或者重写规则，来描述变量间关系
- 通过编译器采用固定算法，使得这些关系产生结果

什么是函数式编程

- 函数式编程是一种编程范式，它是声明式编程的子集
- 避免使用状态、异变对象，最小化副作用
- 基于 lambda 演算，函数可以作为入参和出参

MVC 和 MVVM 的区别是

相同点

- 目的相同：分离模型 Model 和视图 View

不同点

- MVC
 - 构成：模型 Model – 视图 View – 控制器 Controller
 - 分为主动 MVC 和 被动 MVC
 - 主动 MVC：视图订阅数据更新
 - 被动 MVC：控制器操作视图
 - 渲染
 - 后端返回 HTML，利于 SEO
 - 后端返回数据，前端使用模板引擎或操作 DOM
- MVVM
 - 构成：模型 Model – 视图 View – 视图模型 ViewModel
 - ViewModel 单向或双向数据绑定 View 和 Model 层，实现自动同步
 - 渲染
 - 后端减少关心视图，前端 SSR 利于 SEO
 - 前端减少操作 DOM

如何组织 React 项目文件结构？

React 建议

- 项目目录嵌套最多3到4个层级
- 不要过度思考
- 没有官方推荐的组织方式，常见组织方式包括
 - 按功能和路由组织：相同功能或路由的 CSS、JS 和 测试文件放入同一目录
 - 按文件类型组织：组件、页面、API、状态管理、静态文件分类存放

React 18 都有哪些新特性？

- 新的 Root API: ReactDOM.createRoot
 - React 17 及之前版本
 - 通过 ReactDOM.render 将应用渲染到页面的根元素
 - 有限的自动批量处理
 - 如在浏览器事件触发多个状态更改，自动批量更新
 - 而在异步函数中多个状态更改，不会批量更新
 - React 18
 - 可选通过 ReactDOM.createRoot 将应用渲染到页面的根元素

- 确保安全的前提下
 - 如对于每个用户触发的事件，在下一个事件前完成渲染
 - 尽可能地多应用自动批量处理，包括异步函数中的多个状态更新
- 可选 `ReactDOM.flushSync` 退出批量更新
- SSR 支持 `React.lazy` 和 `React.Suspense`
 - React 17 及之前版本
 - `React.lazy()` 和 `React.Suspense` 尚未在 `ReactDOMServer` 中支持
 - React 18
 - 全新的 SSR 架构内置支持 `React.lazy()` 和 `React.Suspense`
- `startTransition`
 - React 17 及以前版本
 - 所有更新都被紧急渲染
 - 使用 `setTimeout` 和 防抖等方式，避免频繁更新
 - React 18
 - 所有渲染分为紧急和非紧急
 - 非紧急渲染使用 `startTransition` 包裹
 - 非紧急渲染的延迟时间由设备决定
 - 非紧急渲染可中断，不会影响响应用户输入、动画等紧急渲染

JSX

什么是 JSX ?

- JSX 是 JavaScript 的语法扩展，生成 React 元素
- JSX 是 `React.createElement(component, props, ...children)` 函数的语法糖
 - React 17 RC 开始，由编辑器自动引入 `import { jsx } from 'react/jsx-runtime'`
- JSX 支持 HTML 模板语法 和 表达式，支持条件和循环渲染，支持点语法和展开运算符
- JSX 转义所有输入内容，防止注入攻击
- JSX 忽略渲染 `false`, `null`, `undefined`, `true` 子元素

为什么推荐在 React 中使用 JSX ?

- React 认为渲染逻辑本质上与其他 UI 逻辑内在耦合
 - UI 中绑定处理事件
 - 状态更新时通知 UI
 - UI 中展示数据
- React 将标记和逻辑共同存放在组件，实现关注点分离
- React 不强制要求使用 JSX，但 JSX 与 UI 一起有视觉辅助作用
- React 通过 JSX 可以显示更多有用错误和警告消息

为什么 JSX 可以有效降低 XSS 风险?

- React DOM 在渲染所有输入内容前，默认会将它们转义成字符串，有效降低 XSS 风险
- 可以通过 dangerouslySetInnerHTML = {{ __html: HTML }} 来显示转义前的内容

如何在 JSX 中条件渲染?

- if / else 语句

```
render() {  
  if (condition) return <div />  
  else return null  
}
```

- 三元运算符

```
render() {  
  return condition ? <div /> : null  
}
```

- 逻辑运算符

```
render() {  
  return condition && <div />  
}
```

- 条件渲染组件

```
render () {  
  const Condition = props => {  
    const { If, children } = props  
    return If && children  
  }  
  return <Condition If={true}><div /></Condition>  
}
```

如何在 JSX 中循环控制?

- map 将数组每一项转成 UI, flatMap filter 等能返回数组的循环方法也被支持

```
render() {  
  const { data } = props  
  return data.map(item => <b id={item.id}>{item.text}</b>)  
}
```

- 使用 JavaScript 的循环（for / while / do while），将结果存储到变量，将变量代入 JSX

```
render() {  
  const { data } = props  
  const datas = Array(data.length)  
  for (let i = 0; i < data.length; i++) {  
    const item = data[i]  
    datas[i] = <b id={item.id}>{item.text}</b>  
  }  
  return datas  
}
```

- 使用 lodash 等第三方库或自定义可以返回数组的方法，以 lodash 为例

```
render() {  
  const { data } = props  
  return _.times(data.length, i => <b id={data[i].id}>{data[i].text}  
</b>)  
}
```

为什么 JSX 中 class 变成了 className？

- JSX 语法上更接近 JavaScript 而不是 HTML
 - HTML 属性值通常为字符串
 - HTML DOM 对象属性值可以是任意数据类型
- JSX 的 className 更接近 HTML DOM 对象的属性，并且支持属性拓展运算符
- JSX 通过匹配闭合标签提升可读性，而不是代替 HTML
 - JSX 与 HTML 需要转换，直接使用 class 也无法避免其它转换工作

组件

什么是 React 组件？

- React 组件允许用户将 UI 拆分成独立可复用的代码片段，并对每个片段进行独立构思
- React 组件从概念上类似于 JavaScript 函数
- React 组件接受任意的入参 Props，返回用于描述页面展示内容的 React 元素

React 组件分成哪几类？

- 按定义分类
 - 类组件，使用 ES6 的 class 定义，维护 state，有生命周期
 - 函数组件，使用普通函数定义，可以通过 hooks 维护状态和副作用
- 按状态分
 - 有状态组件，组件返回结果，受时间、空间或上下文影响
 - 无状态组件，通常是纯展示 UI 组件，容易复用
- 按定位分
 - 展示型组件，接收 props，负责 UI 展示
 - 容器组件，管理 states，负责数据获取和组件间通信，多用于状态提升
- 按 React 内置类型分类
 - 有状态组件
 - ClassComponent，由 class 创建
 - ContextProvider，由 createContext 创建
 - 无状态组件
 - IndeterminateComponent，FunctionComponent 挂载前的初始类型
 - FunctionComponent，即函数组件
 - ForwardRef，由 React.forwardRef 创建，接收 ref 并转发给子组件
 - MemoComponent，由 React.memo 创建，条件渲染子组件
 - SimpleMemoComponent，由 React.memo 创建且不指定条件
 - FiberNode
 - HostRoot，由 ReactDOM.render 创建
 - HostPortal，由 React.createPortal 创建，多用于模态框
 - HostComponent，对应元素节点
 - HostText，对应文本节点
 - 内置类型
 - Fragment，分组子列表，无需向 DOM 添加额外节点，可用短语法<>
 - Profiler，测量 React 应用多久渲染一次以及渲染一次的“代价”
 - StrictMode，严格模式，用来突出显示应用程序中潜在问题的工具
 - Suspense，等待目标代码加载，并且可以指定一个加载界面，在用户等待时显示
 - PureComponent，浅层对比 prop 和 state 实现了 shouldComponentUpdate

类组件和函数组件的区别是？

说明	类组件	函数组件
回调钩子	生命周期	useEffect / useLayoutEffect
this	有，事件处理函数需绑定 this	无

说明	类组件	函数组件
state	有, <code>this.setState</code> 更新	无, <code>useState / useReducer</code> 引入
实例化	是	否
性能	现代浏览器中, 闭包和类的原始性能只有在极端场景才会有明显差别	使用 Hooks 某些情况更加高效, 避免了 class 需要的额外成本, 如创建类实例和在构造函数绑定事件处理器的成本, 符合语言习惯的代码不需要很深的组件库嵌套

受控组件和非受控组件的区别是？

- 受控组件
 - React 的 `state` 是表单元素的“唯一数据源”，控制用户输入过程中表单发生的操作
 - 表单元素的 `value` 跟随 `state` 变化，默认值由 `defaultValue` 设置
 - 表单元素需要被 React 组件包裹
 - 每种数据变化都需要编写事件处理函数
 - 不支持 `value` 只读的表单元素，如 `<input type="file" />` 的 `value` 由用户设置
- 非受控组件
 - 表单数据交由 DOM 节点处理
 - 使用 `ref` 从 DOM 节点获取表单数据
 - 表单元素无需被 React 组件包裹
 - 只关心业务需要的数据变化，减少代码量
 - 集成 React 和非 React 代码，不推荐使用

什么是高阶组件？

- 高阶组件是参数为组件，返回值为新组件的函数，某种角度上就是高阶函数
- 高阶组件是 React 中复用组件逻辑的一种高级技巧
- 高阶组件不是 React API 的一部分，它是一种基于 React 的组合特性而形成的设计模式

什么是 Pure Components？

`React.PureComponent` 与 `React.Component` 相似，区别是

- `React.Component` 并未实现 `shouldComponentUpdate`
- `React.PureComponent` 以浅层对比 `prop` 和 `state` 方式实现了 `shouldComponentUpdate`
 - `React.PureComponent` 无法检查对象的深层差别
 - `prop` 和 `state` 使用深层数据结构时

- 调用 `forceUpdate()` 来确保组件正确更新
- 使用 `immutable` 对象 加速嵌套数据的比较

展示组件和容器组件的区别是？

React 组件按照用途可以分为展示组件和容器组件

React 推荐所有新组件，无论是展示组件，还是容器组件，都采用函数组件 + Hook 方式编写

- 展示组件
 - 关心页面 UI，有自己的 HTML 标签和样式
 - 如果有状态，仅与 UI 相关。与其他组件、store 无关
 - 不关心数据源，通过 `props` 获取数据，并执行回调
- 容器组件
 - 关心功能实现，无自己的 HTML 标签和样式
 - 有状态。包含请求数据源等副作用。状态提升时，维护多个子组件的状态
 - 可以由第三方库生成，如 React Redux 的 `connect()` 和 Relay 的 `createFragmentContainer`

React 分离展示组件和容器组件的优势

- 关注点分离，便于维护
- 提高展示组件的复用度，便于调整 UI
- 便于通过如 `this.props.children` 传递组件本身，减少相同 `props` 层层传递

如何劫持 React 组件提高组件复用度？

劫持 React 组件又被称为渲染劫持

将已有组件包装，注入新属性和功能，输出高阶组件，来实现组件复用

劫持需要遵守高阶组件的约定

- 不要改变原始组件，仅组合组件
- 保持组件的接口与已有组件相似，透传与自身无关的 `props` 给已有组件
- 最大化可组合性，确保函数签名类型一致，输入函数，返回函数，输入组件，返回组件
- 包装显示名称便于调试，如 `withSubscription(CommentList)`

如何设计一个 React 组件？

- 根据数据源和原型和 UI 稿，了解数据结构和 UI 视图
- 划分组件层级
 - 根据单一功能原则分离 UI 与数据源的结构一一对应
 - 明确组件的包含关系
- 构建静态版本
 - 将静态数据通过 `props` 父组件到子组件单向传递

- 构建应用
 - 简单应用，自上而下，从高层组件到低层组件构建
 - 大型应用，自下而上，从低层组件到高层组件构建，同时为低层组件编写测试
- 确定 UI state 最小且完整表示
 - 排除通过 props 传递来的数据
 - 排除不随时间变化的数据
 - 排除可以由其他 state 或 props 计算得出的数据
- 确定 state 放置位置
 - 找出根据 state 渲染的所有组件
 - 找出这些组件的共同上级组件
 - state 应该放置在共同上级组件或者更高层级的组件中
- 添加反向数据流
 - state 只能由拥有它们的组件更改
 - 在该组件添加修改 state 的回调函数
 - 将该回调函数通过 props 传递给子组件，在子组件中，如事件处理函数中调用

React 组件与 Web Components 共存的最佳实践是？

- 访问 Web Components 的命令式 API：使用 ref 与 DOM 节点进行交互
- 引入第三方 Web Components：编写 React 组件包装该 Web Components
- Web Components 触发事件：React 组件中手动添加事件处理器来处理事件

状态

什么是 React 的状态？

- React 的状态 state 是一个对象
 - 类组件中，状态通过 this.state 创建，通过 this.setState 合并更改，异步更新
 - React Hook 中，状态通过 this.useState 或 this.useReducer 使用
- React 将组件看做状态机，状态改变触发渲染
- React 建议减少有状态的组件，提高组件复用度，利于维护
 - 只将无法从 props 传递，无法从其他数据计算，并且随时间可能变化的数据作为 state
 - 多个组件 state 的数据源相同，应将状态提升到父组件或容器组件
 - 避免使用 context，仅在 React 的状态管理无法满足需求时使用 Redux

什么是 React 的状态提升？

- React 中，任何可变数据应当只有一个相对应的唯一“数据源”
- 多个组件反映相同的变化数据时，共享状态提升到最近共同父组件
 - state 应首先添加到需要渲染数据的组件
 - 其他组件也需要这个 state，将它提升至这些组件的最近共同父组件
- state 只能由拥有它们的组件修改，bug 排查范围被大大缩减

状态和属性的区别是什么？

相同点

- state 和 props 都是原生的 JavaScript 对象
- state 和 props 的变化都会触发生命周期、useEffect / useLayoutEffect、和渲染
- state 和 props 相同，渲染结果相同
- state 和 props 都可以在组件内部设置默认值

不同点

- 获取
 - state 由当前组件声明
 - props 由父组件传入
- 更新
 - state
 - 由所在组件通过 useState / useReducer 或 setState 方法更新
 - 由子组件通过 state 所在组件传入的回调函数间接更新
 - props 不允许修改
- 数据
 - state 通常存储可变数据，避免多层嵌套或使用不可变对象便于优化渲染
 - props 除数据外，还多用于回调函数，组件 (children)，路由 (history) 的传递

如何创建动态的状态名称？

- 状态是一个 JavaScript 对象，状态名称即对象的属性名称
- 从 ECMAScript 2015 开始，对象初始化语法开始支持计算属性名。在 [] 放入表达式，计算结果当做属性名。例如

```
const h = {}  
let i = 0  
h[i + 1] = i // h = {'1': 0}
```

- 计算属性名支持对象字面量，例如

```
const { i } = props // props = { i : 0 }
this.state = {
  [i + 1] : i
} // this.state = { '1' : 0 }
```

setState 支持哪些用法？

- `setState(updater, [callback])`
 - `updater` 函数带有形式参数，基于 `state` 和 `props` 构建新对象表示变化
 - `(state, props) => stateChange`，适合后续状态取决于当前状态的情况
- `setState(stateChange[, callback])`
 - `stateChange` 为传入对象，会浅层合并到新的 `state`
 - 同一周期内，后调用的 `setState` 会覆盖先调用的 `setState`
- `setState` 的第二参数为可选回调函数
 - 回调函数将在 `setState` 完成合并并重新渲染组件后执行
 - React 官方推荐使用 `componentDidUpdate()` 生命周期代替 `setState` 的回调函数

setState 和 replaceState 的区别是？

- `setState` 会合并当前状态与之前状态
- `replaceState` 会丢弃之前状态，用新状态替代
- `replaceState` 等同于先在 `setState` 中将状态设置为 `false / null`，再设置新状态

如何优化 setState，减少不必要更新？

- 通过 `setState(updater, [callback])` 的用法，第一参数使用带有形式参数的函数
- 通过 `updater` 函数 `(state, props) => stateChange` 的第一参数，接受原来的 `state` 状态值
- 对比新旧状态值
 - 相同，返回 `null`，不渲染
 - 不同，返回新状态值，触发异步合并渲染

示例代码

```
getData = data => {
  const { time } = data
  this.setState(state => {
    return state.time === time ? null : { time }
  })
}
```

当 State 值为 Object 时，如何优化？

- 避免使用 Object 作为 State 值，使用 PureComponent 的浅比较的自动优化失效
- 必要使用 Object 作为 State 值
 - 避免嵌套过多层级
 - 设置更新 state 的前置条件或使用 shouldComponent 手动优化
- 已经使用 Object 作为 State 值，并且嵌套层级过多
 - 拆分 State 到子组件
 - 使用不可变对象 Immutable，只要 State 更新，返回对象新引用，重新渲染修改节点

属性

什么是 React 的属性？

- 属性是组件的入参，用法同 HTML 自定义属性，可将任意类型数据从父组件传给子组件
- 属性的改变可以触发组件的生命周期流程和渲染
- 建议从组件自身的角度，不依赖于调用中间的上下文命名 Props
- 属性具有只读性，所有 React 组件必须像纯函数一样保护它们的 props 不被更改
- 请避免使用匿名函数作为属性值，避免引起重复渲染
- 具有 render prop 的组件接受一个返回 React 元素的函数，并在组件内部通过调用此函数来实现自己的渲染逻辑

为什么不能直接修改属性？

- React 是“单向”数据流，数据通过 props 传递
- 从 state 派生数据或 UI 只能影响“低于”它们的组件，设计简单高效，便于调试
- 所有 React 组件必须像纯函数一样保护它们的 props 不被更改，保证组件没有副作用

通过属性传递组件本身的方法有哪些？

- props.children
 - 子组件可以获取父组件开始标签和结束标签之间的内容
- render props
 - 向子组件传入函数，返回组件需要渲染什么内容
 - 直接在父组件的标签之间，调用函数返回组件
 - 避免使用匿名函数返回组件，避免重复渲染

使用 key 属性有哪些注意事项？

- key 用来帮助 React 识别哪些元素改变
- key 在数组列表及兄弟节点之间必须唯一
- 不建议使用索引作为 key 值，如果不显示指定 key 值，默认使用索引作为 key 值
- key 只有放在就近的数组上下文中才有意义
- key 不会传递给子组件，需要使用 key 属性的值，需使用其他属性显式传递

如何在 React 中进行静态类型检查？

- React.PropTypes 或 prop-types 库
 - 提供一系列验证器，确保组件接收到的数据类型有效
 - PropTypes 仅在开发模式下进行检查并在控制台显示警告
 - 通过特定的 defaultProps 属性来定义 props 的默认值
- Flow
 - Flow 是一个针对 JavaScript 代码的静态类型检测器
 - Facebook 开发，经常与 React 一起使用
 - Flow 通过特殊类型语法为变量、函数以及 React 组件提供注解
 - Flow 添加方法
 - 将 Flow 添加到项目依赖
 - 确保编译后的代码已经去除 Flow 语法
 - 添加类型注解并且运行 Flow 来检查
- TypeScript
 - TypeScript 是微软开发的编程语言，它是 JavaScript 的类型超集，包含独立编译器
 - 类型语言，构建时可以发现 bug 和错误
 - TypeScript 添加方法
 - 将 TypeScript 添加到项目依赖
 - 配置 TypeScript 编译选项
 - 使用正确的文件扩展名，React 的 JSX 使用 .tsx 作为扩展名
 - 为已经使用的库添加定义，现实其他包的错误和提示
- 新语法和工具链
 - Reason，由 Facebook 开发，经过实战验证的 OCaml 语言
 - Kotlin，由 JetBrains 开发的静态类型语言

如何限制某个属性是必须的？

React 中，可以在任何 PropTypes 属性后加上 `isRequired`，声明该属性必须

- 当必须属性没有被提供值时，控制台打印警告信息
- 表示该属性必须，而不限制属性类型，可以使用 `PropTypes.any.isRequired`
- 表示必须包含一个元素，可以使用 `PropTypes.element.isRequired`

如何设置属性的默认值?

React 中, 通过特定 `defaultProps` 属性来定义 props 默认值

- 设置类组件或函数组件的静态属性 `defaultProps`
- 类组件, 参考 `proposal-class-fields` 提案, 在 `class` 内声明静态属性 `defaultProps`
- 当传入属性值为 `undefined` 时, 使用属性的默认值

React 是否支持 HTML 属性?

- React16 中, 任何标准和自定义的 DOM 属性都是完全支持
- React 为 DOM 提供了一套以 JavaScript 为中心的 API
 - 标准 DOM 属性采用小驼峰命名
 - 自定义属性全部小写
- React 与 HTML 之间部分属性存在差异
 - `checked`
 - 受控组件 `checked`
 - 非受控组件用 `defaultChecked` 设置组件首次挂载时是否被选中
 - `className`
 - 用于指定 DOM 节点和 SVG 元素的 `class`
 - React 中, 使用 Web Components, 使用 `class` 属性代替
 - `dangerouslySetInnerHTML`
 - React 为浏览器 DOM 提供 `innerHTML` 的替换方案
 - 需要向该属性传入 `key` 为 `__html` 的对象, 用来警示跨站脚本 (XSS) 攻击风险
 - `htmlFor`
 - `for` 是 JavaScript 关键字
 - React 元素使用 `htmlFor` 代替
 - `onChange`
 - `onChange` 事件与预期行为一致: 表单字段变化时, 事件都会被触发
 - 与浏览器已有的默认行为不一致:
 - 用户更改 `<input>`, `<select>` 和 `<textarea>` 元素的值并提交更改时 `change` 事件在这些元素上触发
 - 与 `input` 事件不一样, `change` 事件不是每次元素的 `value` 改变时都会被触发
 - React 依靠该事件实时处理用户输入
 - `selected`
 - 将 `<option>` 标记为已选中状态, 请在 `select` 的 `value` 中引用该选项的值
 - `style`
 - 接受小驼峰命名属性的 JavaScript 对象, 而不是 CSS 字符串
 - 与 JavaScript 属性一致, 同时会更高效, 且能预防跨站脚本 (XSS) 的安全漏洞

- 样式不会自动补齐浏览器私有前缀，除 `ms` 外，浏览器引擎前缀都应以大写字母开头
- React 自动添加 `"px"` 后缀到内联样式为数字的属性
 - 本来没有单位的属性 `zoom,order,flex` 不会转换像素字符串
 - 需使用 `"px"` 以外单位，请将值设为数字与所需单位组成的字符串
- `suppressContentEditableWarning`
 - 禁止拥有子节点的元素比标记为 `contentEditable` 时 React 发出警告
- `suppressHydrationWarning`
 - 禁止 React 服务端渲染与客户端渲染不同内容时发出警告
 - 只会对元素一级深度有效，应急方案使用，不过过度使用
- `value`
 - `<input>`, `<select>`, `<textarea>` 组件支持 `value` 属性
 - 非受控组件使用 `defaultValue` 属性设置组件第一次挂载时的 `value`

React 是否支持自定义属性？

- React 16 前会忽略未知的 DOM 属性。JSX 属性 React 无法识别，将被跳过
- React 16 中任何未知的属性都会在 DOM 显示，适用于
 - 非标准属性
 - 尝试实验中的 DOM 接口
 - 集成第三方库或 Web Components

通信

React 父子组件通信有哪些方法？

- `props`
 - 父组件将需要传递的数据，更改 `state` 的方法或组件本身通过 `props` 传递给子组件
 - 子组件通过 `props` 传来的回调函数向父组件传递数据或更改状态
- `refs`
 - 适合在典型数据流之外强制修改子组件的场景，例如
 - 管理焦点，文本选择或媒体播放
 - 触发强制动画
 - 集成第三方 DOM 库
 - DOM `refs` 打破组件封装，`ref` 转发更改开发者预期
 - 应避免过度使用，用声明式实现和状态提升代替
 - 一定要使用时，建议添加注释和说明
 - 被修改的子组件可以是 React 组件实例或 DOM 元素
 - 包含向子组件添加 `ref`、回调 `refs` 和 `ref` 转发三种方式
- `context`

- 适合在组件之间共享如地区偏好，主题等数据，避免逐层传递 props
- 存在 context 的 value 更新，内部所有消费组件都重新渲染问题
- 过度使用组件的状态与上下文相关，复用度降低
- 事件订阅发布
 - 实现并使用 Event Bus
 - 在需要数据的组件监听自定义事件，并传入改变状态更新组件的回调函数
 - 在获取数据的组件触发自定义事件，并传入数据，执行回调函数

```
export default class Eventbus {
  constructor () {
    this.fns = []
  }
  emit (type, ...args) {
    const fns = this.fns[type]
    if (fns) fns.forEach(fn => fn.apply(this, args))
  }
  addListener (type, fn) {
    const fns = this.fns[type]
    fns ? fns.push(fn) : (this.fns[type] = [fn])
  }
}
```

使用 Node.js 的 events

- 使用状态管理，举例说明特点
 - Redux
 - 单一数据源 store
 - state 只能通过 action 更改
 - 使用 reducer 纯函数返回新 state 来更改 state
 - Recoil
 - 任意地方灵活共享 state，并保持高性能
 - 高效可靠地根据变化的 state 进行计算
 - 支持持久化日志
 - 支持 undo
 - hox
 - 只有一个 API
 - 使用 custom Hooks 定义 model
 - 支持 TS
 - 支持多数据源

为什么 React 是单向数据流？

- React 组件不关心其他组件是否有状态和类型
 - 父组件可以把它的 `state` 作为 `props` 向下传递它的子组件中
 - 子组件通过 `props` 接收父组件的数据，不关心数据来源于父组件的 `state` 或 `props`
- React 中这种自上而下的数据传递被称为单向数据流
 - 状态 `state` 总是所属于特定的组件
 - 状态 `state` 派生的任何数据和 UI 只能影响树中“低于”它们的组件
- 单向数据流单向绑定无关，对比双向绑定
 - 坏处：需要绑定更多“样板”代码
 - 好处：排除和隔离 `bug` 所需的工作量将会减少
 - 状态 `state` 只有其所在组件自身可修改
 - `Bug` 可以使用 React 开发者工具来检查问题组件的 `props`
 - 按照组件树结构逐级向上搜寻，直到定位到负责更新 `state` 的组件

什么是 Context？

- Context 提供了一种无需为每层组件手动添加 `props`，能在组件树间进行数据传递的方法
- Context 设计目的是共享或缓存组件树的全局数据，如用户认证，地区偏好，主题和语言
- Context 应用场景是不同层级组件访问同样数据，副作用是降低组件的复用性
- Context API
 - `React.createContext` 创建 Context 对象，订阅该对象的组件从组件树中离自身最近的匹配的 `Provider` 中读取当前的 `context` 值
 - `Context.Provider` 接收 `value` 属性，当其发生变化时，内部所有消费组件都会重新渲染，忽略 `shouldComponentUpdate` 函数
 - `Class.contextType` 订阅单一 `context`，在任何生命周期中，使用 `this.context` 访问
 - `Context.Consumer`
 - 在函数式组件中订阅 `context` 的变更
 - 需要一个函数作为子元素。函数接收当前 `context` 值，并返回一个 React 节点
 - `context` 值由最近的 `Provider` 提供
 - 没有 `Provider` 时等同于 `createContext()` 的 `defaultValue`
 - `Context.displayName` 指定组件在 DevTools 中显示的名称

什么是 ContextType ?

ContextType 用于订阅单一的 context

- 设置 class.contextType 为 React.createContext() 创建的 context 对象

```
const MyContext = React.createContext(defaultValue)
class NewClass extends React.Component {
  render() {
    const value = this.context
  }
}
NewClass.contextType = MyContext
```

- 类组件参考 proposal-class-fields 提案

```
const MyContext = React.createContext(defaultValue)
class NewClass extends React.Component {
  static contextType = MyContext
  render() {
    const value = this.context
  }
}
```

在 React 组件中，使用 this.context 访问通过 contextType 指定的 Context

如何优化 Context ?

- 为什么要优化 Context ?

Context 的 value 更新，它内部所有消费组件都会重新渲染，并且不受制于 shouldComponentUpdate 函数

- 优化 Context 的方法
 - 避免使用对象字面量作为 value

context 使用参考标识 (reference identity) 来决定渲染时机，当 Provider 接收 value 为对象字面量时，每次 value 都会被赋值新对象，建议将 value 状态提升到父节点的 state 里

拆分 Context

将原来同一个 Context 频繁变化的值拆分出来，分别放入不同的 Context

记忆化

使用 React.memo 或 useMemo 包裹组件，指定需要比较的值，只有值变化时重新渲染

使用 `createContext` 的第二参数，不稳定，不推荐

`createContext` 为函数，两个形参分别为更新前后值，接收 `Number` 类型作为返回值

在订阅 `context` 变化的组件上，使用 `unstable_observedBits` 设置重新渲染组件的条件

```
const { Consumer, Provider } = React.createContext(null, (prev, next) => {
  if (prev.value1 !== next.value1) return 1
  if (prev.value2 !== next.value2) return 10
})
const Consumer1 => { // 只有当 value1 变化重新渲染
  return <Consumer unstable_observedBits={1} children={value => (
    <>{value}</>
  )}
  />
}
const Consumer2 => { // 只有当 value2 变化重新渲染
  return <Consumer unstable_observedBits={10} children={value => (
    <>{value}</>
  )}
  />
}
```

使用第三方库

use-context-selector

使用 `createContext` 创建支持 `useContextSelector` 的特殊 `context`

使用 `useContextSelector` 选择 `context` 某个值，当且仅当该值变化时，重新渲染

React Tracke

什么是 Ref 转发？

- Ref 转发可以将 `ref` 传递到子组件
 - 由 `React.forwardRef` 实现
 - 向 `ref` 传入回调函数，函数第一参数是 `React` 组件实例或 `HTML DOM` 元素
- Ref 转发适合应用场景
 - 转发表单组件的 `ref` 到 `DOM` 节点，便于访问 `DOM` 节点，来管理焦点、选中或动画
 - 在高阶组件内，转发外层组件的 `ref` 到 被包裹的组件
- Ref 转发更改了组件默认的 `ref` 指向，对组件使用者不可见，不建议使用
 - 不兼容之前同时使用组件和 `ref` 的应用
 - 对组件使用者，`ref` 结果可能不符合直观预期
- 由 `React.forwardRef` 实现 Ref 转发，可以使用函数决定 `ref` 转发组件显示的内容
 - 设置传入 `React.forwardRef` 函数名称，例如

```
const newComponent = React.forwardRef(function myName(props, ref) {
  return <innerComponent {...props} forwardedRef={ref} />
})
// DevTools DisplayName: ForwardRef(myName)
```

设置函数的 *displayName* 来包含被包裹组件的名称

```
function myName(props, ref) {
  return <innerComponent {...props} forwardedRef={ref}
}
myName.displayName = 'myDisplayName'
const newComponent = React.forwardRef(myName)
// DevTools DisplayName: ForwardRef(myDisplayName)
```

渲染

React 返回空对象有哪些方法？

- false / true / null / undefined 将被忽略，不被渲染

```
render() {
  return false / true / null / undefined
}
```

- React.Fragment / <>

```
render() {
  return <React.Fragment /> / </>
}
```

如何优化不必要的渲染？

- 优化状态 state
 - 避免使用多层嵌套对象作为 state，降低比较效率，提高比较难度
 - 如果超过3层，建议继续拆分组件，将每一层状态放置在更内层的组件中
 - 状态提升和隔离
 - 多个组件需要反映相同的变化数据，共享状态提升到最近的共同父组件
 - 仅影响组件本身的状态隔离放置在组件内部
 - 状态管理库可以使用 Recoil，其使用 Atom 更灵活地管理状态
 - 合并状态更新，分离计算和渲染

- 出于性能考虑，React 可能会把多个 `setState` 调用合并成一个调用
- 如果状态更新后，需要根据新状态计算或判断，再更新状态
 - 那么更新计算前的状态引发的渲染很可能不必要
 - 分离计算逻辑，先计算状态的最终值，直接更新状态到最终值
- 优化属性 `props`
 - 除高阶组件等需保持接口一致的场景外，`props` 保持简单，仅与组件单一职责关联
 - 避免使用多层嵌套对象作为 `props`，降低比较效率，提高比较难度
 - 组件只关心自身相关的 `props`
 - 例如标识单选、复选的选中状态、列表的激活项
 - 为每一项添加是否选中，是否激活标识优于额外添加选中项，激活项列表属性
 - 避免在 `render` 等渲染函数中重复创建对象和函数，先赋值到变量再引用
 - 避免使用对象字面量作为 `props`，每次 `render` 创建新对象，相应子组件重复渲染
 - 避免使用匿名函数作为 `props`，每次 `render` 创建新函数，相应子组件重复渲染
- 先比较再更新
 - 类组件
 - 使用 `PureComponent` 执行一层浅比较 `props` 或 `state`，值变化时渲染
 - 使用 `shouldComponentUpdate` 函数
 - 手动比较前后 `props` 或 `state`，返回 `true` 渲染，`false` 不渲染
 - 若 `state` 为对象，使用不可变对象，简化并加速比较
 - 函数式组件
 - 使用 `React.memo` 比较 `props` 再渲染组件
 - 不使用第二参数时，一层浅比较 `props`，变化时渲染
 - 使用第二参数时，手动比较前后的 `props`，返回 `false` 渲染，`true` 不渲染
 - 使用 `useMemo` 指定依赖，细粒度渲染组件
 - 使用第二参数，传入依赖数组，当数组中的值发生变化时，更新返回值
 - 通常包裹函数式组件的返回值，指定依赖的 `props` 或 `state` 的具体值
 - 包裹任意需要复杂计算的函数，指定依赖的变量。能在同一组件中部分和多次使用
 - 使用 `useCallback` 避免函数重复创建引起渲染
 - 当函数式组件重新更新时，内部声明的函数可能会被重复创建
 - 将这些函数作为 `props` 传入的子组件会重复渲染
 - 使用 `useCallback` 包裹函数，传入空数组，总是返回相同函数来避免重复创建引起的渲染
- 减少嵌套
 - 减少不必要的根组件
 - 返回一个组件列表数组
 - 使用内置组件文档碎片 `React.Fragment` 或其缩写 `<>`

- 谨慎使用 CSS-in-JS 方案
 - Class + 样式表方案的渲染性能和 CSS 本身的复用度总体优于内联方案
 - 避免仅为书写内联样式而频繁地创建嵌套组件
- 惰性渲染
 - key 唯一
 - 使用唯一标识作为 key，而不是 索引，让 React 精确判断变化，复用不变元素
 - 条件渲染
 - 使用 if/else 三元运算符或逻辑运算符，仅在条件满足时，渲染组件
 - 渲染列表时，当条件不满足时，返回 null 或空组件，保留组件位置
 - 防抖和节流
 - 使用 window.requestAnimationFrame 或定时器构建防抖或节流函数
 - 使用 MutationObserver 代替定时器
 - 使用 Intersection Observer 代替 window.scroll
 - 懒加载图片
 - 只渲染可视区域的图片
 - 懒加载 DOM
 - 只渲染可视区域的 DOM，如虚拟列表，商品详情
 - 虚拟列表库可以使用 react-virtualized 及其更新 react-window
 - 异步路由
- 优化 Context，减少 value 更新，内部所有消费组件的重新渲染
 - 拆分 Context
 - 将原属于同一 Context 的值拆分到不同的 Context
 - 记忆化
 - 使用 React.memo 或 useMemo 包裹组件，指定需要比较的值，只有值变化时重新渲染
 - 使用 createContext 的第二参数，不稳定，不推荐
 - createContext 为函数，两个形参分别为更新前后值，接收 Number 类型作为返回值
 - 在订阅 context 变化的组件上，使用 unstable_observedBits 设置重新渲染组件的条件
 - 使用第三方库
 - use-context-selector
 - React Tracke

React 如何渲染 HTML，有什么风险？

- dangerouslySetInnerHTML 是 React 为浏览器 DOM 提供 innerHTML 的替换方案

```
function newComponent() {  
    return <div dangerouslySetInnerHTML={{__html: '<b>1</b>'}} />  
}
```

- 代码直接设置 HTML 存在风险，很容易无意中使用户暴露于跨站脚本（XSS）攻击
- React 设计此替换方案，通过名称警示开发者

React 为什么要引入基于 Fiber 协调器的异步渲染？

- 在经典渲染模式下，React 一旦开始渲染一次更新，不能中断包括创建新 DOM 节点和运行组件中代码在内的工作，这种渲染方法为“阻塞渲染”
- 渲染阻塞响应用户输入和动画，使用防抖牺牲一定的响应即时性，使用节流降低更新频率，都不能提供最佳用户体验。
- React 提供 Concurrent 模式，将人机交互研究的结果整合到真实 UI 中
 - 渲染可中断
 - 有意的加载顺序
 - React 可以在旧屏幕多停留一段时间，跳过不够好的加载状态，直接展示新屏幕
 - 并发，使用启发式方法决定更新紧急性，同时更新多个状态
 - 对于 CPU 密集型更新，例如创建新 DOM 节点和运行组件中代码，任意一个更紧迫更新可以中断已经开始的渲染
 - 对于 IO 密集型更新，例如从网络加载代码或数据，在全部数据到达前在内存中渲染，跳过令人不愉快的空白加载状态
 - 对于超出屏幕显示的更新，延迟它的相关逻辑

什么是 React Fiber？

- React 组件是数据的函数 $v = f(d)$
- 计算机通过调用栈跟踪程序执行
 - 函数执行，栈会添加新堆栈结构，代表函数执行的事务
 - 太多事务同时执行，会导致动画掉帧，用户输入无法及时响应
 - 现代浏览器提供 API
 - requestIdleCallback 调度一个低优先级函数在空闲阶段执行
 - requestAnimationFrame 调度一个高优先级函数在下一个动画帧执行
 - 需要自定义调用栈行为，来使用这些 API 优化 UI 渲染
- React Fiber 是专门为 React 组件实现的堆栈重构
 - 堆栈结构可以保存在内存中，手动调度和操作

- 单个 fiber 是虚拟的堆栈结构，对应一个组件实例
 - 包含组件及其输入、输出信息的 JavaScript 对象
 - type 是执行会被堆栈结构跟踪的函数
 - 复合函数、组件类 class 或字符串
 - key 决定协调算法 Fiber 是否可重用
 - child 指向
 - 类组件 class 的 render 方法返回值
 - 函数组件的 return 返回值
 - sibling 指向
 - 类组件 class 或函数组件返回数组，其中包含多个子节点的情况
 - return 返回父 fiber
 - 属性，即函数参数，以下两者相等，则 fiber 可复用
 - pendingProps fiber 开始时设置
 - memoizedProps fiber 结束时设置
 - pendingWorkPriority 事务优先级数字
 - 0 表示 NoWork
 - 数字越大，优先级越低
 - 调度器使用该字段搜索下一个要执行的事务
 - alternate
 - flush 渲染输出结果到屏幕
 - work-in-progress 进行中的 fiber 代表未返回堆栈结构的 fiber
 - output
 - host component 是 React 应用的叶子节点，浏览器环境是小写 HTML 标签
 - 每个 fiber 输出结果由宿主组件创建，向上传递到整个树

React Fiber 异步渲染分为哪几个阶段，对应生命周期是什么？

不同于 Stack reconciler，Fiber reconciler 过程分为两个阶段：

- render / reconciliation 可中断阶段
 - 以 Fiber tree 为蓝本，每个 Fiber 为工作单元，自顶向下构造 workInProgress Tree
 - 判断节点是否需要更新
 - 需要更新，标记当前节点
 - 更新当前节点状态和属性
 - 调用 shouldComponentUpdate
 - 返回 true 需要更新
 - 调用 render

- 返回子节点，并创建对应 fiber
 - 无子节点
 - 当前工作单元结束，返回副作用列表
 - 检查 fiber 的 sibling 或 child
 - 存在，作为下一个工作单元
 - 检查剩余可用时间
 - 有，立即开始下一个工作单元
 - 无，等待下一次主线程空闲，开始下一个工作单元，通过 requestIdleCallback 实现
 - 没有下一个工作单元，本阶段结束，进入 pendingCommit 状态
- commit 不可中断阶段
 - 执行副作用列表
 - 更新 DOM 树，更新 ref 指向
 - 调用组件的生命周期函数或 useEffect 声明的钩子

对应生命周期

- render / reconciliation
 - componentWillMount
 - componentWillReceiveProps
 - shouldComponentUpdate
 - componentWillUpdate
- commit
 - componentDidMount
 - componentDidUpdate
 - componentWillUnmount

生命周期

React 组件有哪些生命周期方法？

React 每个组件都包含“生命周期”方法

开发者可以重写这些方法，便于在运行过程中特定阶段执行这些方法

- constructor(props)
 - React 组件挂载之前调用
 - 用于初始化 state 和 方法绑定，例如绑定事件处理函数的 this
- static getDerivedStateFromProps(props, state)
 - React 组件调用 render 方法前，初始挂载及后续更新都会被调用
 - 用于返回一个对象来更新 state，返回 null 不更新任何内容
 - 每次渲染都会触发，包括父组件重新渲染以及组件本身调用 setState

- 以下场景不使用 `getDerivedStateFromProps`
 - 执行副作用应用 `componentDidUpdate`
 - `prop` 更改时重新计算某些数据，请使用 `memoization helper` 代替
 - `prop` 更改时“重置”某些 `state`
 - 请考虑使组件完全受控或使用 `key` 使组件完全不受控 代替
- `shouldComponentUpdate(nextProps, nextState)`
 - `props` 或 `state` 发生变化时，渲染执行之前被调用
 - 首次渲染及 `forceUpdate()` 调用，`React` 参考返回 `true / false`，决定是否重新渲染
- `render()`
 - `class` 组件中唯一必须实现的方法，纯函数，每次调用应返回相同结果
 - 当 `render` 被调用时，它会检查 `this.props` 和 `this.state` 变化，并返回
 - `React` 元素：通过 `JSX` 创建
 - 数组或 `frgements`：使得 `render` 方法可以返回多个元素
 - `Portals`：可以渲染子节点到不同 `DOM` 子树
 - 字符串或数值类型：可以在 `DOM` 中被渲染为文本节点
 - 布尔类型或 `null`：什么都不渲染
- `getSnapshotBeforeUpdate(prevProps, prevState)`
 - 在最近一次渲染输出（提交到 `DOM` 节点）之前调用
 - 能在组件发生更改之前从 `DOM` 中捕获信息，例如滚动位置
 - 此生命周期方法的任何返回值将作为参数传递给 `componentDidUpdate` 的第三个参数
- `componentDidMount()`
 - 组件挂载后（插入 `DOM` 树中），即首次渲染时，立即调用
 - 适用于放置依赖 `DOM` 节点、网络请求获取数据、添加订阅等
 - 可以直接调用 `setState()`，触发依赖于 `DOM` 节点大小或位置的渲染
- `componentDidUpdate(prevProps, prevState, snapshot)`
 - 组件更新后立即调用，首次渲染不会执行此方法
 - 可以执行 `DOM` 操作，比较前后 `props`，网络请求
 - 可以直接调用 `setState()`，它必须被包裹在一个条件语句里，避免死循环
 - 如果组件实现了 `getSnapshotBeforeUpdate()` 生命周期，则生命周期的返回值将作为 `componentDidUpdate` 的第三个参数
- `componentWillUnmount()`
 - 组件卸载及销毁之前直接调用
 - 用于执行必要的清理操作，清除定时器、取消网络请求以及在 `componentDidMount()` 中创建的订阅等
 - 不应调用 `setState()`，避免组件永远不会重新渲染

React 组件的生命周期可分为哪些阶段？

React 组件的生命周期可以分为三个阶段

- "Render"阶段：纯净且不包含副作用，可能会被 `React` 暂停、中止或重新启动

- constructor(props)
- getDerivedStateFromProps(props, state)
- shouldComponentUpdate(nextProps, nextState)
- render()
- "Pre-commit"阶段：可以读取DOM
 - getSnapshotBeforeUpdate(prevProps, prevState)
- "Commit"阶段：可以使用 DOM，运行副作用，安排更新
 - 挂载时：componentDidMount()
 - 更新时：componentDidUpdate(prevProps, prevState, snapshot)
 - 卸载时：componentWillUnmount()

异步数据请求应在哪些生命周期里调用？

- 通过网络请求获取数据或订阅数据更新
 - componentDidMount()
 - 原因
 - 首次挂载时执行
 - 不会重复请求或订阅数据更新
 - 无需条件不容易触发重复渲染
- 有条件地通过网络请求数据
 - componentDidUpdate(prevProps, prevState, snapshot)
 - 原因
 - 可对比更新前后的 props 或 states
 - 设置条件
 - 当且仅当条件满足时，通过网络请求数据
 - 避免触发重复渲染
- 取消网络请求或者清除在 componentDidMount() 中创建的订阅
 - componentWillUnmount()

useEffect useLayoutEffect 与生命周期的对应关系是？

useLayoutEffect

- 执行时机是组件挂载或更新之后，浏览器执行绘制之前
 - 与 componentDidMount + componentDidUpdate 一致
- 支持返回清除函数，函数执行时机是组件卸载之前
 - 与 componentWillUnmount 一致

useEffect

- 执行时机是组件挂载或更新之后，浏览器完成布局和绘制之后，在一个延迟事件中被调用
 - 与 `componentDidMount + componentDidUpdate` 不同
- 支持返回清除函数，函数执行时机是组件卸载之前
 - 与 `componentWillUnmount` 一致

两者都适用于在函数组件主体内，即 React 渲染阶段改变 DOM，添加订阅，设置定时器，记录日志以及执行其他包含副作用的操作

优先使用 `useEffect` 避免阻塞视觉更新，只在需要读取 DOM 布局，在浏览器绘制前，同步触发重渲染的场景使用 `useLayoutEffect`

在 constructor 中使用 super 的意义是？

- JavaScript 中，`super` 指向父类构造函数，React 类组件可以是 `React.Component` 等
- JavaScript 中，调用 `super` 前不能用 `this`。React 中不能声明 `this.state` 或读取 `this.props`
- React 中，调用 `super(props)` 后，可以在 `constructor` 中访问 `this.props`
- React 中，调用 `super()` 不传参，不能在 `constructor` 中访问 `this.props`
 - React 在调用构造函数后，将 `props` 赋值到实例
 - 可以在其他生命周期中，访问 `this.props`
- 不调用 `super`，参考 `proposal-class-fields` 提案，可以将 `state` 等实例属性写在 `class` 内

```
class MyComponent extends React.Component {  
  state = { time : 0 }  
}
```

对比 React Hook 与生命周期

- 什么是 React Hook
 - React Hook 是 React 16.8 的新增特性
 - 开发者可以在不编写 `Class` 的情况下使用 `State` 以及其他的 React 特性
- 为什么要用 React Hook 代替拥有生命周期的类组件
 - 类组件的弊端
 - 组件之间复用状态逻辑很难
 - React 没有提供将可复用性行为“附加”到组件的途径
 - 使用 `render props` 和高阶组件
 - 需要重新组织组件结构，使得代码难以理解
 - 形成“嵌套地域”，难以调试和维护
 - 复杂组件变得难以理解
 - 组件被状态逻辑和副作用充斥

- 不相关逻辑被放在同一生命周期，相关逻辑被分散在不同生命周期
 - `componentDidMount/componentDidUpdate` 获取数据
 - `componentDidMount` 可能还包含事件监听，需在 `componentWillUnmount` 清除
- 大量的状态逻辑，许多人使用状态管理库，组件复用和调试都比较困难
- 难以理解的类组件
 - 大量的 `this` 绑定，较高学习成本，区分函数组件和类组件的使用场景
 - 类组件会无意中鼓励开发者使用一些让 `Prepack` 等组件预编译优化措施无效的方案
 - 类组件不能被很好的压缩，热重载不稳定，实例化类组件有轻微额外性能消耗
- 有限的是否渲染控制
 - 使用 `PureComponent` 执行一层浅比较 `props` 或 `state`，值变化时渲染
 - 使用 `shouldComponentUpdate` 函数
 - 手动比较前后 `props` 或 `state`，返回 `true` 渲染，`false` 不渲染
 - 若 `state` 为对象，使用不可变对象，简化并加速比较
 - 在 `setState` 前设置判断条件，满足条件后再 `setState`
- React Hook 解决的问题
 - 提高组件复用
 - Hook 在无需修改组件结构的情况下，复用状态逻辑，共享 Hook 变得更便捷
 - 简化组件
 - Hook 将组件中相互关联的部分拆分成更小的函数（比如设置订阅或请求数据）
 - 非按照生命周期划分，可以使用 `Reducer` 来管理组件内部状态，使其更加可预测
 - 函数组件代替类组件
 - Hook 在不使用类组件的情况下使用 `State` 以及其他的 `React` 特性
 - Hook 改造类组件为函数组件，容易理解 `this` 和学习，便于预编译等优化措施
 - Hook 拥抱函数式声明式编程，而无需学习复杂的函数式或响应式编程技术
 - 丰富的更细粒度的是否渲染控制
 - 使用 `React.memo` 比较 `props` 再渲染组件
 - 不使用第二参数时，一层浅比较 `props`，变化时渲染
 - 使用第二参数时，手动比较前后的 `props`，返回 `false` 渲染，`true` 不渲染
 - 使用 `useMemo` 指定依赖，细粒度渲染组件
 - 使用第二参数，传入依赖数组，当数组中的值发生变化时，更新返回值
 - 通常包裹函数式组件的返回值，指定依赖的 `props` 或 `state` 的具体值

- 包裹任意需要复杂计算的函数，指定依赖的变量。能在同一组件中部分和多次使用
- 使用 `useCallback` 避免函数重复创建引起渲染
 - 当函数式组件重新更新时，内部声明的函数可能会被重复创建
 - 将这些函数作为 `props` 传入的子组件会重复渲染
 - 使用 `useCallback` 包裹函数，传入空数组，总是返回相同函数来避免重复创建引起的渲染

事件处理

React 和 DOM 事件处理的区别是？

React 元素的事件处理和 DOM 元素的不同点

- 事件命名
 - React: 小驼峰式
 - DOM: 纯小写
- 事件处理函数传参
 - React: 接受函数
 - DOM: 字符串
- 阻止默认行为
 - React: 显式的使用 `preventDefault`
 - DOM: `return false`
- 事件处理函数，第一参数 `e`
 - React: `e` 是一个合成事件。React 根据 W3C 规范来定义合成事件
 - DOM: `e = window.event`
- 添加事件监听
 - React: 元素初始渲染的时候添加监听器
 - DOM: 使用 `addEventListener`
- `this` 指向
 - React: 在 `class` 组件中，手动 `bind this`，使用匿名箭头函数或者 `class fields` 语法来让 `this` 指向组件本身
 - DOM: 触发事件的目标对象
- 向事件处理程序传递参数
 - React: 通过箭头函数和 `Function.prototype.bind` 来实现
 - DOM: 经常通过 DOM `attribute` 传参，读取参数时，先获取目标对象，再获取目标对象的 `attribute`
- 事件委托
 - React: 16 及更早版本，对大多数事件执行 `document.addEventListener()` 17 后调用 `rootNode.addEventListener()`
 - DOM: 自行添加事件委托

什么是 React 合成事件?

- React 合成事件是浏览器的原生事件的跨浏览器包装器
 - 兼容所有浏览器
 - 接口与浏览器原生事件相同，包括 `stopPropagation()` 和 `preventDefault()`
 - 合成事件实例将被传递给事件处理函数
- 事件池
 - React 16 及更早版本，合成事件是合并而来，放入事件池统一管理
 - 事件对象可能会被重用
 - 在事件回调函数被调用后，所有属性失效
 - 不能通过异步访问事件属性
 - 异步访问事件属性
 - 调用 `e.persist()`，从事件池中移除合成事件，允许用户代码保留对事件的引用
 - 通过实例的 `nativeEvent` 属性来使用浏览器的底层事件
 - React 17 合成事件不再放入事件池
 - `e.persist()` 存在，但不再生效
 - 可以通过异步访问事件属性
- 事件执行顺序
 - 原生事件先执行
 - 合成事件后执行
 - 默认事件处理函数在冒泡阶段触发
 - 注册捕获阶段事件处理函数，应为事件名添加 `Capture`
- 合成事件会冒泡
 - React 16 及更早版本，冒泡绑定到 `document` 上
 - React 17，冒泡到 `rootNode` 上，`onScroll` 事件不再冒泡

如何在 React 事件处理阻止默认行为?

调用 `e.preventDefault` 阻止默认行为

如何解决 类组件 中，事件处理的 this 为 undefined 的问题?

- 使用 `Function.prototype.bind`，添加事件处理函数时，给函数绑定 `this`
- 使用 `public class fields` 语法，可以使用 `class fields` 正确的绑定回调函数
- 使用 匿名回调函数 添加事件，不推荐，原因是
 - 匿名回调函数成为组件的 `prop` 传入子组件，可能导致额外的重新渲染

如何传参给事件处理函数？

两种方式

- 箭头函数
 - 声明一个匿名箭头函数，在其中调用事件处理函数并传参
 - 不推荐，原因是：
 - 匿名回调函数成为组件的 `prop` 传入子组件，可能导致额外的重新渲染
- `Function.prototype.bind`
 - `bind` 将返回一个新函数，新函数的 `this` 是 `bind` 的一个参数，其余参数将作为函数的参数，供调用时使用

如何阻止事件处理函数被频繁调用？

- 节流
 - 一定时间内一定调用一次事件处理函数
 - 核心区别：定时器不重置
 - 连续重复调用，定时器不重置，只有第一次调用生效，忽略后面调用
 - 手写代码
 - 原生

```
const throttle = (fn, delay) => {
  let timer = null
  return function (...args) {
    if (timer) return
    timer = setTimeout(() => {
      timer = null
      fn.apply(this, args)
    }, (delay + '') | 0 || 1000 / 60)
  }
}
```

原生 + *React Hook*

```
const useThrottle (fn, delay) => {
  const ref = useRef(fn)
  useEffect(() => {
    ref.current = fn
  })
  return useCallback(
    throttle(...args) => ref.current(...args), delay),
    [delay]
  )
}
```

- 防抖
 - 一定时间内最多调用一次事件处理函数
 - 核心区别：定时器重置
 - 连续重复调用，定时器重置，只有最后一次调用生效，忽略前面调用
 - 手写代码
 - 原生

```
const debounce = (fn, delay) => {
  let timer = null
  return function (...args) {
    if (timer) clearTimeout(timer)
    timer = setTimeout(() => {
      timer = null
      fn.apply(this, args)
    }, (delay + '') | 0 || 1000 / 60)
  }
}
```

原生 + *React Hook*

```
const useDebounce (fn, delay) => {
  const ref = useRef(fn)
  useEffect(() => {
    ref.current = fn
  })
  return useCallback(
    debounce(...args) => ref.current(...args), delay),
    [delay]
  )
}
```

React 17 对事件处理做了哪些改进？

- 更改事件委托
 - 更加安全地进行新旧版本 React 树嵌套，多版本都必须为 17 或更高版本
 - React 16 及之前版本
 - 向 document 附加事件处理器
 - 大多数事件执行 document.addEventListener
 - React 17
 - 向 React 树的根 DOM 容器附加事件处理器
 - 大多数事件执行 rootNode.addEventListener
 - React 嵌入使用其他技术构建的应用程序变得更加容易
 - React 16 及之前版本
 - e.stopPropagation() 将被多 React 版本破坏，嵌套树结构中阻止了事件冒泡，但外部树依然能接收到它
 - e.stopPropagation() 不会阻止其它技术构建应用的冒泡
 - React 17 的事件冒泡更接近常规 DOM，更符合预期
 - e.stopPropagation() 将阻止多 React 版本的事件冒泡，包括前套树和外部树
 - e.stopPropagation() 将阻止其它技术构建应用的冒泡
- 对标浏览器
 - React 16 及之前版本
 - onScroll 事件冒泡，onFocus 事件冒泡
 - onFocus 和 onBlur 事件底层由 React 实现
 - 捕获事件（例如 onClickCapture）由 React 实现
 - React 17 与浏览器行为更接近，并提高了互操作性
 - onScroll 事件不再冒泡
 - onFocus 和 onBlur 事件底层切换为原生的 focusin 和 focusout 事件
 - 捕获事件（例如 onClickCapture）现在使用的是实际浏览器中的捕获监听器
- 去除事件池
 - React 16 及以前版本
 - 使用 event pooling（事件池）
 - 重用不同事件的事件对象，以提高性能
 - 所有事件字段会被重置为 null，必须调用 e.persist() 才能正确使用事件，或者读取需要的属性
 - React 17 去除事件池，事件对象行为符合预期
 - 去除事件池
 - 不再重用事件对象，避免对现代浏览器的负优化
 - 所有事件对象不会被重置为 null，无需调用 e.persist()，使用和读取属性与原生事件一致。e.persist() 为兼容而保留，已无实际作用

样式管理

如何在 React 中使用样式?

- Style
 - style 属性接收含有驼峰命名的 JavaScript 对象
 - 属性名与 JS DOM API 一致，防止 XSS
 - React 自动添加 px 后缀到内联样式为数字的属性之后
 - style 多用于在渲染过程中添加动态计算的样式
 - 从性能角度来说，通常不推荐将 style 属性作为设置元素样式的主要方式，应使用 className 属性来引用外部 CSS 样式表中定义的 class
- Class
 - className 支持字符串
 - className 支持大括号包裹的 JS 表达式
- CSS-in-JS
 - CSS 由 JavaScript 生成而不是在外部文件中定义
 - 常用的库包括 styled-components emotion radium 和 aphrodite 等
 - React 对样式如何定义没有明确态度
 - 存在疑惑时，比较好的方式是使用 *.css 定义样式，通过 className 指定它们
- 动画
 - 渐变效果，可选 React Transition Group
 - 动画效果，可选 React Motion 和 React Spring

如何按条件加载样式?

- Style

style 采用小驼峰命名属性的 JavaScript 对象，可以按照 JavaScript 对象的方式，有条件地改变属性名和属性值

```
const style = {
  [2 > 1 ? 'fontSize' : 'lineHeight'] : 2 > 1 ? 12 : 1
} // { fontSize: 12 }
function Component () {
  return <div style={style}>Hello World!</div> // style="font-size: 12px"
}
```

- Class

className 属性，可以传入 JavaScript 表达式，有条件改变的类名

```
const className = 2 > 1 ? 'menu-active' : 'menu'
function Component () {
  return <div className={className}>Hello World!</div> // class="menu-active"
}
```

- CSS-in-JS

以styled-components 为例

模板字符串

```
const Button = styled.button`
background: ${props => props.primary ? 'palevioletred' : 'white'},
` // background: palevioletred
function Component () {
  return <Button>Hello World!</Button>
}
```

JavaScript 对象

```
const Button = styled.div(props => ({
  background: props.primary ? 'palevioletred' : 'white'
})); // background: palevioletred
function Component () {
  return <Button>Hello World!</Button>
}
```

如何合并多个内联样式?

style采用小驼峰命名属性的 JavaScript 对象

任何合并 JavaScript 对象的方法都可以用于合并内联样式

- 拓展运算符

```
const backgroundStyle = { background: 'red' }
const colorStyle = { color: 'red' }
function Component () {
  return <div style={{ ...backgroundStyle, ...colorStyle }}></div>
}
```

- Object.assign

```
const backgroundStyle = { background: 'red' }
const colorStyle = { color: 'red' }
function Component () {
  return <div style={Object.assign(backgroundStyle, colorStyle)}>
</div>
}
```

- 遍历赋值

```
const backgroundStyle = { background: 'red' }
const colorStyle = { color: 'red' }
Object.keys(colorStyle).forEach(key => backgroundStyle[key] =
colorStyle[key])
function Component () {
  return <div style={backgroundStyle}></div>
}
```

如何模块化样式，如何避免样式名冲突？

(1) 什么是 CSS 模块化？

CSS 模块化是将 CSS 规则 拆分成相对独立的模块，便于开发者在项目中更有效率地组织 CSS

- 管理层叠关系
- 避免命名冲突
- 提高复用度，减少冗余
- 便于维护或扩展

CSS 模块化的方式

- 基于文件拆分
- 不拆分但设置作用域
- CSS in JS
- 内联样式、Shadow DOM 等

无论哪种方式，核心都是通过 保证CSS类命名唯一，或者 避免命名使用内联样式，来模拟出 CSS模块作用域的效果

(2) 基于文件的 CSS 模块的加载

- <link>
将不同模块的 CSS 分文件存放，通过 标签按需引入
- @import
@规则，将其它 CSS 嵌入当前 CSS
除现代浏览器外，也得到了 CSS 预处理器 Less、Sass 和 Stylus 的支持
- import
在 Webpack 中，将 CSS 作为资源引入，通过 CSS Modules 生成独一无二的类名

(3) CSS 模块化的实现方式

- 分层拆分

将 CSS 规则 分层存放，并约束不同层次间的命名规则

- SMACSS: 按功能分成 Base Layout Module State Theme 五层
- ITCSS: 按从通用到具体分成 Settings Tools Generic Base Objects Components 七层

- 分块拆分

将页面中视觉可复用的块独立出来，仅使用类选择器，并确保类名唯一

- OOCSS
 - 将盒模型规则与颜色、背景等主题规则拆分
 - 将视觉可复用的块类名与页面结构解耦
- BEM
 - 将页面按 Block Element Modifier 划分
 - 类名规则 `block-name__element-name--modifier-name`

- 原子化拆分

每个选择器只包含 1 个或少量属性，通过组合选择器定义复杂样式

- ACSS: 属性名像函数，属性值像函数参数，像写内联样式一样组合类名
- Utility-First CSS: 提供一套可配置的 CSS 实用类库，使用时按需编译

- CSS in JS

- CSS Modules
 - 将 CSS 作为资源引入
 - 根据内容的哈希字符串，计算出独一无二类名，CSS 规则 只对该类名下的元素生效
- styled-components Aphrodite Emotion 等
 - 通过 JS 创建包含属性定义的组件
 - 生成独一无二的类名
- Radium 等
 - 通过 JS 创建包含属性定义的组件
 - 生成内联样式

- Shadow DOM

通过attachShadow给元素的影子DOM，附加<style>标签，其中规则不会影响外部元素。代表的框架有 Ionic 等

(4) React 中的样式模块化

React 对样式如何定义，没有明确的态度，如果存在疑虑，比较好的方式是和平常一样，在一个单独的 *.css 定义你的样式，并通过 className 指定它们。此时，你可以参考 CSS 模块化的方案，通过文件或样式命名规则，来实现样式模块化

行内样式 style 是避免样式命名冲突的经典策略，但由于性能比 className 低，React 通常不推荐将 style 属性作为设置元素样式的主要方式。在多数情况下，应使用 className 属性来引用外部 CSS 样式表重定义的 class。style 在 React 应用中多用于在渲染过程中添加动态计算的样式

同样地，在 CSS-in-JS 中，早期有基于 style 实现的 Radium 库，现在比较流行的是基于随机 className 的 style-components 库，与之类似的有 Emotion 和 Glamor，还有面向愿意将 CSS 和 JavaScript 分开存放开发者的 JSS 等

错误边界

什么是 React 错误边界？

(1) 为什么要有错误边界？

部分 UI 的 JavaScript 错误不应该导致整个应用崩溃，为了解决整个问题，React 16 引入了错误边界的概念

(2) 什么是错误边界？

错误边界是一种 React 组件，这种组件可以捕获发生在其子组件树任何位置的 JavaScript 错误，并打印这些错误，同时展示降级 UI，而并不会渲染那些发生崩溃的子组件树。

错误边界在渲染期间、生命周期方法和整个组件树的构造函数中捕获错误。

(3) 如何构建一个错误边界组件？

如果一个 class 组件中定义了 `static getDerivedStateFromError()` 或 `componentDidCatch()` 这两个生命周期方法中的任意一个（或两个）时，那么它就变成一个错误边界，其中

- `static getDerivedStateFromError()` 用于渲染备用 UI
- `componentDidCatch()` 用于打印错误信息

使用只需用错误边界组件包裹 React 组件

错误边界可以捕获什么错误？

错误边界可以捕获子组件的错误，包括子组件

- 渲染期间
- 生命周期方法
- 以及整个组件树的构造函数

以上其中的错误

错误边界不能捕获什么错误？

错误边界不能捕获以下场景产生的错误：

- 事件处理

- 异步代码
 - 定时器
 - `requestAnimationFrame` / `requestIdleCallback` 的回调函数
 - `Promise`
 - Rxjs 的 `Subscribe` 的 回调函数
 - `Observer`，例如 `IntersectionObserver` / `MutationObserver`
- 服务端渲染
- 自身抛出来的错误

错误边界应该放置在什么位置？

错误边界的位置通常由开发者决定，可以用错误边界组件包裹

- 最顶层的路由组件：一旦有组件报错，展示错误信息
- 单独的组件：当组件报错时，保护应用其他部分不崩溃

组件的哪些生命周期可以用于错误捕获？

组件有两个声明周期方法可以用于错误捕获

- `static getDerivedStateFromError(error)`
 - 此生命周期会在后代组件抛出错误后被调用
 - 它将抛出的错误作为参数，并返回一个值以更新 `state`
 - 此生命周期会在渲染阶段调用，因此不允许出现副作用

`componentDidCatch(error, info)`

此生命周期在后代组件抛出错误后被调用

它将抛出的错误作为第一参数 `error`

它将带有 `componentStackkey` 对象，即包含有关组件引发错误的栈信息作为第二参数 `info`

此生命周期会在提交阶段调用，允许执行副作用，如打印、记录或上报错误日志等

React 如何处理未捕获错误，为什么这样处理？

(1) React 如何处理未捕获错误

自 React 16 起，任何未被错误边界捕获的错误将会导致整个 React 组件树被卸载。在浏览器环境使用 `ReactDOM` 渲染的页面将展示为白屏，并在控制台输出报错信息

(2) 为什么这样处理？

React 认为把一个错误的 UI 留在那比完全移除它更糟糕。在即时通信应用中展示错误的消息，在支付类应用中展示错误的金额，都比不呈现任何内容更糟糕

白屏有助于开发者发现已存在但未曾注意到的崩溃，手动增加错误边界，让开发者应用发生异常时提供更好的用户体验

React 推荐使用 JS 错误报告服务，了解在生产环境中出现的未捕获异常，并将其修复

如何处理事件处理函数内部错误？

错误边界无法捕获事件处理函数内部的错误，因为事件处理不会在渲染期间触发，即使异常，React 仍然能够知道需要在屏幕上显示什么

捕获事件处理函数内部的错误，使用 JavaScript 的 `try / catch` 语句即可

事实上，我们可以用 `window.addEventListener('error', () => {})` 捕获大部分同步、定时器、Generator 异步错误，用 `window.addEventListener('unhandledrejection')` 捕获 Promise 及其语法糖 `async / await` 错误

Hook

什么是 React Hook？

Hook 是 React 16.8 的新增特性

- 允许开发者在函数组件里“钩入”React state 及生命周期等特性的函数
 - React 内置如 `useState`、`useEffect` 等 Hook
- Hook 将组件中相互关联的部分拆分成更小的函数（比如设置订阅或请求数据）
 - 非强制按照生命周期划分，避免每个生命周期包含不相关的逻辑
- 开发者可以在不编写 `class` 的情况下使用 `state` 以及其他的 React 特性
 - 无需考虑 `this`，无需考虑函数和 `class` 组件的区别和应用场景
 - 便于使用 `Prepack` 试验 `component folding`，使代码更易于优化
 - 拥抱函数式编程
- Hook 和现有代码可以同时工作，渐进式地使用

什么是 State Hook？

- State Hook 允许开发者在 React 函数组件中添加 `state` 的 Hook
- 这种 Hook 在 React 的原生实现是 `useState`，它是一种函数调用时保存变量的方式，它与 `class` 里面的 `this.state` 提供的功能完全相同
- `useState` 的唯一参数是初始 `state`，支持数字、字符串、对象等类型
- `useState` 方法的返回值分别是当前 `state` 以及更新 `state` 的函数，使用数组解构获取赋值
- `state` 只在组件首次渲染时创建，下次重新渲染时，返回当前的 `state`

什么是 Effect Hook?

- Effect Hook 允许开发者在函数组件中执行副作用操作，包括数据获取、设置订阅以及手动更改 React 组件中的 DOM。分为需要清除和不需要清除的
- 这种 Hook 在 React 的原生实现是 `useEffect`
- `useEffect` 支持两个参数
 - 第一个参数为函数，默认在组件第一次渲染和每次更新、浏览器完成画面渲染之后执行，调用时机可以看作 `class` 组件的 `componentDidMount`、`componentDidUpdate` 生命周期 + 浏览器完成画面渲染时
 - 第二个参数为数组，只有数组元素发生变化时，才会调用第一个参数的函数
 - 当第二个参数为空数组 `[]` 时，第一个参数的函数仅会在组件挂载和卸载时执行
- `useEffect` 可以返回一个函数，用于移除订阅等副作用，区别 `class` 组件的 `componentWillUnmount`，React 会在执行当前 `effect` 之前对上一个 `effect` 进行清除
- `useEffect` 放在最底层内部可以直接访问 `state` 和 `props`

如何清除 Effect Hook 的副作用?

- 在 `useEffect` 的 `effect` 函数中，返回一个清除函数
 - 可以将添加和清除副作用的逻辑放在一起
- `effect` 在每次渲染的时候会执行，
 - React 会在执行当前 `effect` 之前对上一个 `effect` 进行清除

使用 Hook 需要遵循的规则是?

Hook 本质是 JavaScript 函数，使用 Hook 时需要遵循两条规则

- 只在最顶层使用 Hook
 - 不要在循环，条件或嵌套函数中调用 Hook
 - 确保总在 React 函数的最顶层调用 Hook
 - 确保 Hook 在每一次渲染都按照相同的顺序被调用
 - 多次的 `useState` 和 `useEffect` 调用中间保持 Hook 状态正确
- 只在 React 函数中调用 Hook
 - 仅在 React 的函数组件，而不要在普通的 JavaScript 函数中调用 Hook
 - 在自定义 Hook 中调用其他 Hook

如何校验 Hook 是否遵循规则？

React 提供了一个名为 `eslint-plugin-react-hooks` 的 ESLint 插件来校验 Hook 是否遵循规则

- 安装

```
npm install eslint-plugin-react-hooks -D
```

- ESLint 配置

```
{
  "plugin": [
    // ...
    "react-hooks"
  ],
  "rules": {
    // ...
    "react-hooks/rules-of-hooks": "error", // 检查 Hook 规则
    "react-hooks/exhaustive-deps": "warn" // 检查 effect 依赖
  }
}
```

useMemo 和 useCallback 的区别是？

useMemo

```
const memoizedValue = useMemo(() => computeExpensiveValue(a, b), [a, b])
```

返回一个 `memoized` 值，把 "创建" 函数和依赖项数组作为参数传入 `useMemo`，它仅会在某个依赖项改变时才重新计算 `memoized` 值。这种优化有助于避免在每次渲染时都进行高开销计算。

useCallback

```
const memoizedCallback = useCallback(() => { doSomething(a, b) }, [a, b])
```

返回一个 `memoized` 回调函数，把内联回调函数及依赖项数组作为参数传入 `useCallback`，它将返回该回调函数的 `memoized` 版本，该回调函数仅在某个依赖项改变时才会更新。

`useCallback` 可看作函数组件的 `shouldComponentUpdate`，使用引用相等性避免非必要渲染

`useCallback` 返回的是函数，`useMemo` 返回的是值，也可以是函数

`useCallback(fn, deps)` 相当于 `useMemo(() => fn, deps)`

useReducer 和 useState 的区别是？

useReducer

```
const [state, dispatch] = useReducer(reducer, initialArg, init)
```

useReducer 是 useState 的替代方案，它接收一个形如 `(state, action) => newState` 的 reducer，返回当前的 state 及其配套的 dispatch 方法

useReducer 比 useState 更适用于某些场景

- state 逻辑较复杂，并且包含多个子值
- 下一个 state 依赖于之前的 state

useReducer 第二个返回值是 dispatch 而不是回调函数

- 可以向子组件传递 dispatch 而不是函数
 - 容易会触发深更新的组件做性能优化

useLayoutEffect 和 useEffect 的区别是？

useLayoutEffect

```
useLayoutEffect(() => {  
  // add effect  
  return cleanup() {  
    // clean effect  
  }  
})
```

相同

- useLayoutEffect 的签名与 useEffect 相同
- 服务端渲染时，useLayoutEffect 和 useEffect 都无法在 Javascript 代码加载完成前执行
 - 服务端渲染组件引入 useLayoutEffect 触发 React 告警，解决
 - 使用 useEffect 替代 useLayoutEffect
 - 或将该组件延迟到客户端渲染完成后再显示

区别

- useLayoutEffect
 - 在所有 DOM 变更之后同步调用 effect，用来读取 DOM 布局并同步触发重渲染
 - 在浏览器执行绘制之前，useLayoutEffect 内部的更新计划将被同步刷新
- useEffect
 - 在所有 DOM 变更并且浏览器完成渲染后调用 effect
 - 尽可能使用标准的 useEffect 以避免阻塞视觉更新

- 只有 `useEffect` 有问题时，再尝试 `useLayoutEffect`

useRef 和 Refs 的区别是？

useRef

```
const refContainer = useRef(initialValue)
```

`useRef` 返回一个可变的 `ref` 对象，其 `current` 属性被初始化传入参数 `initialValue`
返回的 `ref` 对象在组件的整个生命周期保持不变

Refs

```
class Component extends React.Component {
  constructor(props) {
    super(props)
    this.myRef = React.createRef()
  }
  render() {
    return <div ref={this.myRef} />
  }
}
```

`Refs` 使用 `React.createRef()` 创建的，并通过 `ref` 属性附加到 `React` 元素
当 `ref` 被传递给 `render` 中的元素时，对节点的引用可以在 `ref` 的 `current` 属性中被访问

区别

useRef

- 函数组件可用
- 用途多样，`useRef()` 创建一个普通 Javascript 对象，每次渲染时返回同一个 `ref` 对象
 - 可以在其 `current` 属性保存任何可变值。

Refs

- `class` 组件或 `HTML` 元素，函数组件无实例，不可用
- 用途单一，`.current` 属性为实例的引用，根据节点的类型不同
 - 当 `ref` 属性用于 `HTML` 元素时，构造函数中使用 `React.createRef()` 创建的 `ref` 接收底层 `DOM` 元素作为 `current` 属性
 - 当 `ref` 属性用于自定义 `class` 组件时，`ref` 对象接收组件的挂载实例作为其 `current` 属性

如何自定义 Hook?

(1) 自定义 Hook 定义

自定义 Hook，是将组件逻辑提取到可重用的函数，它可以像 render props 和高阶组件来共享组件之间的状态逻辑，而不增加组件

(2) 如何自定义 Hook

自定义 Hook 是一个函数，可以调用其他的 Hook

- 名称以 "use" 开头，表示这是一个 Hook
 - 便于 React 判断函数内部是否包含对其内部 Hook 的调用
 - 自动检查 Hook 是否违反 Hook 规则
- 参数和返回可以自定义，可以像使用函数一样在不同 Hook 间传递信息
- 其中所有 state 和副作用完全隔离，每次调用 Hook，都会获取独立的 state

将两个函数组件的共有的状态逻辑提取到自定义 Hook 中，自定义 Hook 是一种自然遵循 Hook 设计的约定，而不是 React 的特性

(3) 自定义 Hook 作用

自定义 Hook 解决了以前在 React 组件中无法灵活共享逻辑的问题

自定义 Hook 可以用于表单处理、动画、订阅声明、计时器等场景

尽量避免过早地增加抽象逻辑，当函数组件代码行数增多时，可以通过自定义 Hook 简化代码逻辑，解决组件杂乱无章

是否有必要使用 Hook API 重写所有类组件?

不，React 推荐称为编写 React 组件的主要方式，并且提供了自定义 Hook 等 class 组件无法实现的功能，更利于自动代码优化。但是

- Hook 暂时不能覆盖 class 组件的所有场景，比如生命周期 `getSnapshotBeforeUpdate`，`getDerivedStateFromError` 和 `componentDidCatch` 暂无 Hook 等价写法
- 没有计划从 React 中移除 class

所以，React 鼓励写新组件的时候开始尝试 Hook，不推荐用 Hook 重写已有的 class，除非开发者本来就打算重写它们（例如：为了修复 bug）

useState 返回更新 state 的函数是同步，还是异步的？

useState 返回更新 state 函数是异步的，接收参数可以是新状态的值，也可以是回调函数。可以通过两种方式，获取上一个 state 的状态值

- 回调函数的第一个参数是上一个状态的值，可以像使用 useReducer 的 reducer 函数一样，基于上一个 state 来生成新的 state
- 或者可以在 useEffect 或 useLayoutEffect 中获取更新后的状态，将 useEffect 或 useLayoutEffect 封装到自定义 Hook 同样有效

测试

什么是 Jest？

Jest 是一款 JavaScript 测试运行器，零配置，支持快照，并行，迭代快，生态丰富

- Jest 支持模拟或者真实的 web 标准环境
 - 支持 jsdom 模拟各种 web 标准的 JavaScript 实现，用于 Node.js 提供仿真 DOM 环境
 - 支持与 Puppeteer 等 headless 浏览器协作，在真实的 web 环境中测试 React 组件
- Jest 拥有丰富的 Mock Functions API，可以模拟 modules 和 timers 精细控制代码执行
- Jest 提供足够的 Exceptions 方法，测试失败时，并提供丰富的上下文信息
- Jest 只需添加 --coverage 参数，即可生成完整的测试覆盖率报告

如何模拟数据获取？

- 可以防止由于后端不可用导致的测试不稳定
- 加快测试运行速度

模拟数据获取

- jest.mock

```
import axios from 'axios'
jest.mock('axios')
test('Test axios', async () => {
  axios.get.mockResolvedValueOnce('data')
  const data = await axios.get('/api')
  expect(data).toBe('data')
})
```

- jest.spyOn

```
test('Test fetch', async () => {
  jest.spyOn(global, 'fetch').mockImplementation(() =>
    Promise.resolve({
      json: Promise.resolve('data')
    })
  )

  const response = await fetch('/api')
  const data = await response.json()
  expect(data).toBe('data')
})
```

如何模拟组件？

可以用 `jest.mock` 来模拟组件

- `jest.mock` 第二参数为函数
- 函数接收 `props` 作为参数，返回一个函数组件

```
jest.mock('./common/MyComponent', () => props => <>{props.text}</>)
```

如何模拟计时器？

- 使用 `jest.useFakeTimers()` 开启全局模拟计时器
 - 模拟 `setTimeout`, `setInterval`, `clearTimeout`, `clearInterval`
- 使用 `jest.runAllTimers()` 快进，直到所有计时器都被执行
- 使用 `jest.runOnlyPendingTimers()` 快进，直到当前被挂起的计时器都被执行，不包含在此过程中创建的任何新计时器，避免递归计时器死循环
- 使用 `jest.advanceTimersByTime(milliseconds)` 快进指定毫秒数，直到毫秒数之前的计数器都被执行

如何模拟浅层渲染？

浅层渲染即只渲染父组件而不渲染所有子组件，可以只渲染组件的“第一层”

- 方便对组件的 `render` 方法的返回值进行断言
- 不必担心子组件的行为，子组件没有实例化或被渲染，浅层渲染不依赖 DOM

浅层渲染的实现

组件：MyCompoent.js

```
import React from 'react'
const MyComponent = () => <><div></div></>
```

- 基于 Enzyme

```
import { shallow } from 'enzyme'
import MyComponent from './MyComponent'
describe('shadow render', () => {
  const myComponent = shallow(<MyComponent />)
  expect(myComponent.contains(<div></div>)).toEqual(true)
})
```

- 基于 react-test-renderer

```
import ShallowRenderer from 'react-test-renderer/shallow'
const renderer = new ShallowRenderer()
renderer.render(<MyComponent />)
const result = renderer.getRenderOutput()
expect(result.props.children).toEqual([<div></div>])
```

如何将组件渲染成 JS 对象？

react-test-renderer 与 jest 的快照功能一起用于简化 React 组件测试

```
import renderer from 'react-test-renderer'
test('Link renders correctly', () => {
  const tree = renderer.create(
    <Link page="http://www.facebook.com">Facebook</Link>
  ).toJSON()
  expect(tree).toMatchSnapshot()
})
```

- 第一次运行测试，Jest 将创建快照文件

```
exports[`Links renders correctly 1`] = `
<a
  className="normal"
  href="http://www.facebook.com"
  onMouseEnter={[Function bound _onMouseEnter]}
  onMouseLeave={[Function bound _onMouseLeave]}
  Facebook
</a>
```

- 之后的测试，Jest 将简单地将渲染输出与之前的快照进行比较
 - 如果匹配，通过测试
 - 如果不匹配
 - 需要更新快照 `jest -u`
 - 测试失败

如何模拟 DOM 环境?

测试通常在无法访问真实渲染表面（如浏览器）的环境中运行

建议使用 jsdom 来模拟浏览器，这是一个在 Node.js 内运行的轻量级浏览器实现

- 大多数情况下，jsdom 行为类似常规浏览器，但不具备布局和导航的功能，大多数基于 Web 的组件测试仍然有用，因为它的运行比每个测试启动浏览器的方式效率更高
- 它与你编写的测试运行在同一个进程，可以用来检查和断言渲染 DOM
- jsdom 模拟用户交互，测试可以在 DOM 节点上派发事件，观察并断言这些操作的副作用

综上，可以使用 Jest 作为测试运行器，渲染到 jsdom，使用 act() 辅助函数提供的能力通过一系列的浏览器事件来模拟用户交互行为，此外

- mocha 可以模拟布局及真实输入等原生浏览器行为，用于测试浏览器的特定行为
- 无法模拟 DOM 环境，需要模拟事件交互
 - 使用事件模拟辅助函数
 - 使用 @testing-library/react-native 中的 fireEvent 辅助函数
- 使用真实浏览器渲染整个应用，可以使用
 - Cypress
 - puppeteer
 - webdriver

如何获得测试代码覆盖率?

前端 javascript 测试代码覆盖率的常用工具为 istanbul

使用 jest 框架，可以传入参数 --coverage 获得测试代码覆盖率

如何测试 React Router?

测试 React Router，需要先使用 createMemoryHistory() 来创建导航历史，模拟点击导航按钮，等待页面 DOM 加载，判断应加载页面的独有元素是否被加载

以 @testing-library/react 为例

App.jsx

```
import React from 'react'
import { Switch, Route, Link } from 'react-router-dom'
export App = () => {
  return (
    <>
    <nav>
    <Link data-testid="home-link" to="/">Home</Link>
    <Link data-testid="about-link" to="/about">About</Link>
    </nav>
```

```

    <Siwtch>
    <Route path="/" component={Home} /> // contains <img id="home" />
    <Route path="/about" component={About} /> // contains "About
Company" text
    </Siwtch>
  </>
)
}

```

App.test.js

```

import React from 'react'
import { Router } from 'react-router-dom'
import { render, fireEvent } from '@testing-library/react'
import { createMemoryHistory } from 'history'
import App from './App'
import '@testing-library/jest-dom/extend-expect'

const renderRouter = Component => {
  const history = createMemoryHistory()
  return render(<Router history={history}>{Component}</Router>)
}

describe('App Test', () => {
  it('render home page', () => {
    const { getByTestId } = renderRouter(App)
    expect(getByTestId('home')).toBeInTheDocument()
  })
  it('render about page', () => {
    const { container, getByTestId } = renderRouter(App)
    fireEvent.click(getByTestId('about-link'))
    expect(container.innerText).toHaveTextContent('About Company')
  })
})

```

原理

什么是 Virtual DOM ?

Virtual DOM 是一编程概念

- UI 以一种理想化的，或者说“虚拟的”表现形式被保存在内存中
 - 支持可以优化的 Diff 算法
- 通过 ReactDOM 等类库使之与“真实的” DOM 同步，这一过程叫做协调
 - 支持优先级，并行可中断的协调策略
 - 支持 ReactCanvas 和 ReactNative 等其他渲染方式，甚至非浏览器环境

Virtual DOM 赋予 React 声明式的 API

- 告诉 React 希望让 UI 是什么状态，React 就确保 DOM 匹配该状态
- 开发者不必关心属性操作、事件处理和手动 DOM 更新这些构建应用程序必要的操作

Virtual DOM 是一种模式，在 React 中

- Virtual DOM 通常与 React 元素关联，代表用户界面的对象
- fibers 内部对象来存放组件树的附加信息
- React Fiber 是协调引擎，主要目的是使 Virtual DOM 可以增量式渲染

什么是 React Diff，对比 Vue Diff？

虚拟 DOM 的 Diff 算法

- 将新旧虚拟 DOM 看作两棵节点树，节点个数为 n
 - 左侧树的节点需要与右侧树的节点一一对比，需要 $O(n^2)$ 复杂度
 - 删除未找到的节点，需要再找合适节点放到被删除位置，需要 $O(n)$ 复杂度
 - 添加新节点，需要 $O(n)$ 复杂度
- 综上，Diff 虚拟 DOM 的复杂度是 $O(n^3)$

React 基于以下两个假设的基础之上提出 $O(n)$ 的启发式算法

- 两个不同类型的元素会产生不同的树
- 可以通过设置 **key** 属性，来告知渲染哪些子元素在不同的渲染下可以保存不变

React Diffing 算法

- Tree Diff
 - 对比两棵树时，首选比较两棵树的根节点。不同类型的根节点元素会有不同的形态
 - 根节点为不同类型的元素
 - React 会拆卸原有的树并且建立起新的树
 - 当卸载一棵树时
 - 对应的 DOM 节点会被销毁
 - 建立一棵新的树时，对应的 DOM 节点会被创建以及插入到 DOM 中
 - 组件实例将执行 `componentWillUnmount()` 方法
 - 根节点为相同类型的元素
 - React 会保留节点
 - 仅比对及更新有改变的属性
 - 处理完根节点，React 继续对子节点进行递归
- 通过 `updateDepth` 控制 Virtual Dom 树层级
- 只比较同一个父节点的子节点
- 通过删除和创建节点实现跨层级移动
- 避免跨层级移动，优先 CSS 控制显示隐藏

- Component Diff
 - 同类型组件
 - 组件实例会保持不变，因此可以在不同的渲染时保持 `state` 一致
 - `React` 将更新该组件实例的 `props` 以及保证与最新的元素保持一致
 - 调用该实例的 `UNSAFE_componentWillReceiveProps`、`UNSAFE_componentWillUpdate()` 以及 `componentDidUpdate()` 方法
 - 调用 `render()` 方法，`diff` 算法将在之前的结果以及新的结果中进行递归
 - 通过 `shouldComponentUpdate`、`useMemo`、`useCallback` 手动优化
 - 不同类型组件，输出内容相似
 - 建议改成同一类型，避免重新渲染组件
 - 不同类型组件
 - 删除和创建
- Element Diff
 - 默认情况下，`React` 会同时遍历两个子元素的列表，递归 `DOM` 节点的子元素
 - 产生差异时，生成一个 `mutation`
 - 用 `key` 标识节点
 - 避免使用索引 `index`，而应使用例如 `id` 唯一标识来作为 `key`
 - 只顺序移动位置变到前面的节点
 - 相同类型 `React` 元素，保留 `DOM` 节点，仅对比及更新改变的属性

Vue 2.x 优化 Diff 算法

- 基本优化与 `React` 相同
- `pathNode`
 - 新老节点相同，不更新
 - 新老节点都是静态节点，`key` 相同
 - `新节点.elm = 老节点.elm`
 - `新节点.componentInstance = 老节点.componentInstance`
 - 新老节点存在，不相同
 - 用 `updateChildren` 更新
- `updateChildren`
 - 虚拟 `DOM` 双指针，真实 `DOM` 双指针，一一对应
 - 两端到中间，直到虚拟 `DOM` 或真实 `DOM`，左指针 > 右指针

Vue 3.x 优化 Diff 算法

- 创建 `VNode` 确定类型，内容不会变化的 `DOM` 添加静态标记
- 在 `mount / patch` 中用位运算判断 `VNode` 类型
 - 静态提升 `hoistStatic`
 - 不参与更新的元素，只创建一次，渲染时直接复用

- 事件侦听器缓存 `cacheHandlers`
 - 缓存函数，不追踪变化，提升性能

什么是 React Concurrent 模式？

Concurrent 模式是一组 React 的新功能，可帮助应用保持响应，并根据用户的设备性能和网速进行适当的调整

Concurrent 模式的特点包括

- 可中断渲染
 - 中断一项正在执行的更新去做更重要的事情，然后再回到之前正在做的工作
 - 减少防抖和节流在 UI 中的需求。因为渲染可以终端，不需要人为地延迟工作以避免卡顿
- 有意的加载顺序
 - React 首先在内存中准备新屏幕
 - 之前，React 继续显示完全互动，带有内联加载指示器的旧屏幕
 - 当新屏幕准备就绪之后，React 可以带我们跳转到新屏幕
- 并发
 - 在 Concurrent 模式中，React 可以同时更新多个状态
 - 对于 CPU 密集更新（例如创建新的 DOM 节点和运行组件中的代码），并发意味着一个更急迫的更新可以“中断”已经开始的渲染
 - 对于 IO 密集的更新（例如从网络加载代码或数据），并发意味着 React 甚至可以在全部数据到达之前就在内存中开始渲染

Concurrent 模式的任务是帮助将人机交互研究的结果整合到真实的 UI 中

- 避免在屏幕之间切换时显示过多的中间加载状态
- 对于用户输入、悬停、点击、页面转换等在内部使用不同的“优先级”，大致对应于人类感知研究中的交互类别

Concurrent 模式的开启

- 安装 React 实验版本

```
npm install react@experimental react-dom@experimental
```

- 开启 Concurrent 模式
 - blocking 模式：提供了 concurrent 模式的小部分功能，更接近于当前 React 的 legacy 模式
 - `ReactDOM.createBlockingRoot(rootNode).render(<App />)`
 - concurrent 模式：未来 React 默认开发模式
 - `ReactDOM.createRoot(rootNode).render(<App />)`

Concurrent 模式常用 API

- **Suspense**
 - 允许组件在渲染之前进行“等待”，并在等待时显示 **fallback** 的内容
- **SuspenseList**
 - 通过编排向用户显示这些组件的顺序，来帮助协调血多可以挂起的组件
- **useTransition**
 - 允许组件在切换到下一个界面之前等待内容加载，从而避免不必要的加载状态
 - 允许组件将速度较慢的数据获取更新推迟到随后渲染，以便能够立即渲染更重要的更新
- **useDeferredValue**
 - 返回一个延迟响应的值，该值可能“延后”的最长时间为 **timeoutMs**
 - 适合立即渲染用户输入内容，同时需要等待数据获取的内容，保持接口可响应性的场景

什么是 Suspense ?

- **Suspense** 让组件“等待”某个异步操作，直到该异步操作结束即可渲染
- **Suspense** 不是一个数据请求的库，而是一个机制
 - 这个机制是用来给数据请求库向**React**通信说明某个组件正在读取的数据当前仍不可用
 - 通信之后，**React**可以继续等待数据的返回并更新**UI**
- **Suspense**不是什么
 - 不是数据获取的一种实现
 - 不是直接用于数据获取的客户端
 - 不使数据获取与视图层代码耦合
- **Suspense**可做什么
 - 让数据获取库与**React**紧密整合
 - 让开发者有针对性地安排加载状态的展示
 - 它能够消除 **raceconditions**，避免由于代码运行顺序的错误假设而导致的bug
 - 不是等到响应报文被接收之后，才去更新 **State**
 - 反过来，发出请求之后，马上更新 **State**（外加开始渲染）
- **Suspense** 可以不必等到数据全部返回才开始渲染
 - 一发送网络请求，马上开始渲染
 - 读取的数据没获取完毕，组件会处于“挂起”状态
 - **React** 会跳过“挂起”组件，继续渲染组件树中的其他组件
 - 随着更多数据的到来，**React** 将尝试重新渲染，并且每次都可能渲染出更加完整的组件树

React 如何定义任务的优先级?

任务优先级是产生更新对象之后，React去执行一个更新任务，这个任务的优先级

任务优先级被用来区分多个更新任务的紧急程度，对比前后两次更新的任务优先级

- 后者 > 前者，React会取消前者的任务调度
- 后者=前者，React 会将同等优先级的更新收敛到一次任务中
- 后者 < 前者，React会在前者更新完成后，再对后者发起任务调度

简而言之，任务优先级存在的意义

- 保证高优先级任务及时响应
- 收敛同等优先级的任务调度

React定义的任务优先级分为三类

- 同步优先级：React的 **legacy** 同步渲染模式产生的更新任务的优先级
- 同步批量优先级：React的 **blocking**模式产生的更新任务的优先级
- **Concurrent** 模式优先级：对于用户输入、悬停、点击、页面转换等在内部使用不同的“优先级”，大致对应于人类感知研究中的交互类别

LanePriority 越大，优先级越高

```
export const SyncLanePriority: LanePriority = 17
export const SyncBatchedLanePriority: LanePriority = 16
const InputDiscreteHydrationLanePriority: LanePriority = 15
export const InputDiscreteLanePriority: LanePriority = 14
const InputContinuousHydrationLanePriority: LanePriority = 13
export const InputContinuousLanePriority: LanePriority = 12
const DefaultHydrationLanePriority: LanePriority = 11
export const DefaultLanePriority: LanePriority = 10
const TransitionShortHydrationLanePriority: LanePriority = 9
export const TransitionShortLanePriority: LanePriority = 8
const TransitionLongHydrationLanePriority: LanePriority = 7
export const TransitionLongLanePriority: LanePriority = 6
const RetryLanePriority: LanePriority = 5
const SelectiveHydrationLanePriority: LanePriority = 4
const IdleHydrationLanePriority: LanePriority = 3
const IdleLanePriority: LanePriority = 2
const OffscreenLanePriority: LanePriority = 1
export const NoLanePriority: LanePriority = 0
```

Redux

什么是 Redux

Redux 是 JavaScript 应用的可预测状态容器

- 可预测
 - 在不同环境（客户端、服务器和移动端）行为一致，易于测试
- 集中
 - 集中应用状态和逻辑，支持撤销、重做和持久化
- 可调试
 - Redux DevTools 可以方便地跟踪应用的状态何时、何处、为什么及如何改变
 - Redux 架构允许开发者记录更改，使用“时间旅行调试”，向服务器发送完整错误报告
- 灵活
 - Redux 可用于任何 UI 层，并拥有一个庞大的插件生态系统来满足您的需求

Flux 和 Redux 的区别是？

Flux

- Flux 是用于构建用户界面的应用程序架构，通过单向数据流补充 React 可组合的视图组件
- Flux 更像模式而非框架，没有任何硬依赖
- Flux 架构的应用包含 4 部分
 - Action
 - 通过 Action creators 创建
 - 每个 Action 拥有 type 或类似属性
 - 传递给 Dispatcher
 - Dispatcher
 - 分发 Actions 给所有注册 Store 的回调函数
 - Store
 - 接受 Action 更新数据后，触发 change 事件，通知 View
 - 可以由多个 Store
 - View 视图组件，即 Controller-View
 - 监听 change 事件，从 Store 取回数据，将数据传递给子组件或更新组件状态
 - 响应用户输入，生成新的 Action

Redux

- Redux 是 JavaScript 应用的可预测状态容器

- Redux 对不同框架都有完整实现，Facebook 官方推荐使用代替 Flux
- Redux 架构与 Flux 基本一致，但做了简化
 - State 只读，更改 State 的方式是返回新的对象，即引入 Reducer 纯函数
 - Action 与 Dispatcher，只需返回包含 type 和 payload 属性的对象
 - Store
 - 唯一
 - createStore 基于 Reducer 纯函数创建
 - store.dispatch() 调用 Action
 - View
 - 通过 store.getState() 获取最新状态
 - 通过 store.subscribe() 订阅状态更新
 - store.subscribe() 返回函数可取消订阅

综上，Redux 与 Flux 都基于单向数据流，架构相似，但 Redux 默认应用只有唯一 Store，精简掉 Dispatcher，引入 Reducer 纯函数，通过返回新对象，而不是更改对象，更新状态。

对比 Flux 的官方实现，Redux 的 API 更简洁，并且提供了如 combineReducers 等工具函数及 React-Toolkit 工具集，以及对状态的撤销、重做和持久化等更复杂的功能。提供如 React-Redux 等简化 Redux 与其他第三方库的连接。

Facebook 官方推荐在生产环境中使用代替 Flux。

Redux 的核心原则是？

Redux 设计和使用遵循三个基本原则：

- 单一数据源，Store 唯一
 - 整个应用程序的状态 State 存储在单一对象树 Object tree 中
 - Object tree 只存在唯一的 Store 中
 - 单一对象树让跟踪状态的时间变化，调试和检查应用程序都更加容易
- 状态是只读的
 - Redux 假设开发者永远不会更改数据，而是在 Reducer 中返回新对象来更新状态
 - 更改状态的唯一方法是发出一个动作 Action，Action 是对已发生事情的抽象描述的对象
 - 数据变更，如用户输入和网络请求都不能直接更改状态
- Reducer 是纯函数，用来归并状态 State
 - 接受原状态和 Action，返回新状态 reducer(state, action) => new State
 - 纯函数，无副作用，输出和输入一一对应，与执行上下文，时间，调用次数无关。不应在函数内请求 API，操作 DOM，使用 Date.now() 等时间耦合方法或随机值

React Context 和 Redux 的区别是？

ReactContext

- React Context API 是为了解决跨组件层级传递 props 的效率问题
- 试验性的 Context API 存在问题
 - 提供数据源的父组件和接收数据的子组件间的某个组件的 `shouldComponentUpdate` 返回 `false` 跳过更新，子组件也会被动跳过更新
- Context API 正式在 React16.3 引入，使用方法
 - 创建 context 对象实例

```
xxxxxxxxx const MyContext = React.createContext({ Provider, Consumer }) =  
MyContext
```

使用 `<Provider value={{`v`}`a`lu`}`e}>` 组件去声明想要传递的数据源

消费数据

高阶组件写法 `<Consumer>{{`v`}`aule=> ... }</Consumer>`

Hook 写法 `const value = useContext(MyContext)`

Redux

- Redux 是 JavaScript 应用的可预测状态容器
- Redux 使用方法
 - 声明 reducer 函数

```
const reducer = (state, action) => {  
  switch (action.type) {  
    case 'INCREMENT': return state + 1  
    case 'DECREMENT': return state - 1  
    default: return state  
  }  
}
```

创建 store 对象

```
import { createStore } from 'redux'  
const store = createStore(reducer)
```

使用 `state` 和 `dispatch`

```
const MyComponent = ({ value, onIncrement, onDecrement }) => (  

```

```

    <>
      {value}
      <button onClick={onIncrement}>+</button>
      <button onClick={onDecrement}>-</button>
    </>
  )
  const App = () => (
    <MyComponent
      value={store.getState()}
      onIncrement={() => store.dispatch({type: 'INCREMENT'})}
      onDecrement={() => store.dispatch({type: 'DECREMENT'})}
    />
  )

```

区别

- 目的
 - **ReactContext** 解决跨组件层级传递 **props** 的效率问题
 - **Redux** 是 JavaScript 应用的可预测状态容器，拥有完整的状态管理功能
- 更新机制
 - **ReactContext**: Context 的 **value** 更新，它内部所有消费组件都会重新渲染，并且不受制于 **shouldComponentUpdate** 函数，需要手动优化
 - 避免使用对象字面量作为 **value**
 - 拆分 Context
 - 记忆化
 - 使用 **createContext** 的第二参数手动优化
 - **Redux**
 - 只有当前组件所消费的状态值改变，当前组件才会被更新
- 调试
 - **ReactContext** 支持 **ReactDevTools** 调试
 - **Redux** 支持 **Redux DevTools** 调试
 - 可以方便地跟踪应用的状态何时、何处、为什么及如何改变
 - **Rddux** 架构允许开发者记录更改，使用“时间旅行调试”
- 中间件
 - **ReactContext** 不支持中间件
 - **Redux** 支持 **applyMiddleware** 将所有中间件组成一个数据，依次执行，最后执行 **store.dispatch**，依靠中间件扩展 **Redux** 功能，如简化异步操作等

React 访问 ReduxStore 的方法有哪些？

- **connect**: 适合 React 组件访问 **ReduxStore**

```
import React from 'react'
import { connect } from 'react-redux'
const MyComponent = ({value}) => <>{value}</>
const mapStateToProps= ({value}) => ({value})
export connect(mapStateToProps)(MyComponent)
```

- 导出 **store**: 适合非服务端渲染
 - 创建store并导出

```
import { createStore } from 'redux'
import reducer from './reducer'
const store = createStore(reducer)
export default store
```

引入store 通过 *getState* 获取状态

```
import store from './store'
export function get(){
  const state = store.getState()
  const { value } = state
}
```

- 使用 **redux-thunk** 的第二参数 *getState*

```
import { createStore, applyMiddleware } from 'redux'
import thunk from 'redux-thunk'
import reducer from './reducer'
const store = createStore(reducer, applyMiddleware(thunk))
const get = () => (dispatch, getState) => {
  dispatch({type: 'getStart'})
  const { token } = getState()
  fetch('/user/info', {
    method: 'GET',
    header: {
      Authorization: `Bearer ${token}`
    }
  }).then(res => res.json()).then(json => {
    dispatch({type: 'getSuccess', payload: {user: json} })
  })
}
store.dispatch(get())
```

- 手写中间件 **middleware**, 截获 **action** 的 **payload**, 或者直接输出 *getState* 方法

```
import { createStore, applyMiddleware } from 'redux'
import reducer from './reducer'
const payload = null
```

```
const myMiddleware = store =>next =>action=>{
  if (typeof action === 'function') {
    return action(store)
  }else{
    payload= action.payload
  }

  return next(action)
}
const store = createStore(reducer, applyMiddleware(myMiddleware))
store.dispatch(store=> store.getState())// 直接调用 store的方法
store.dispatch({
  type: 'start',
  payload : 'payload'
})
console.log(payload) // payload
```

Redux 中异步请求数据时发送多 Action 方法有哪些?

异步请求数据等异步操作通常要发出三种 Action

- 操作发起时 Action, 以 start 为例
- 操作成功时 Action, 以 success 为例
- 操作失败时 Action, 以 failure 为例

发送多 Action 方法

- mapDispatchToProps

```
const mapDispatchToProps = dispatch=>{
  return {
    asyncAction () {
      dispatch({ type: 'start' })
      fetch('/api').then(res => res.json()).then(data=> {
        dispatch({type: 'success',payload: { data } })
      }).catch(error=> {
        dispatch({ type: 'failure',payload: { error } })
      })
    }
  }
}
```

- redux-thunk, 使 dispatch支持传入函数


```
import { createStore, applyMiddleware } from 'redux'
import thunk from 'redux-thunk'
import reducer from './reducer'
const store = createStore(reducer, applyMiddleware(thunk))
const asyncAction = dispatch => {
  dispatch({ type: 'start' })
  fetch('/api').then(res => res.json).then(data => {
    dispatch({ type: 'success', payload: { data } })
  }).catch(error => {
    dispatch({ type: 'failure', payload: { error } })
  })
}
store.dispatch(asyncAction())
```

- **redux-promise**, 使 **dispatch** 支持传入 **Promise**

```
import { createStore, applyMiddleware } from 'redux'
import promiseMiddleware from 'redux-promise'
import reducer from './reducer'
const store = createStore(reducer, applyMiddleware(promiseMiddleware))
const asyncAction = dispatch => fetch('/api').then(res
=>res.json).then(data => {
  dispatch({ type: 'success', payload: { data } })
}).catch(error => {
  dispatch({ type: 'failure', payload: { error } })
})
store.dispatch({ type: 'start' })
store.dispatch(asyncAction())
```

- **redux-saga**, 采用 **Generator** 生成器函数支持异步多 **Action**
 - **sagas.js**

```
import { call, put, takeEvery, takeLatest } from 'redux-saga/effects'
function* asyncAction() {
  try{
    const data= yield call('/api')
    yield put({ type: 'success', payload: { data } })
  } catch(error){
    yield put({ type: 'failure', payload: { error } })
  }
}
function* mySaga() { // 支持并发
  yield takeEvery('start', asyncAction)
}
function* mySage() { //不支持并发, 前个处理中的相同 type 的 Action 会被取消
  yield takeLatest('start', asyncAction)
}
export default mySage
```

```
import { createStore, applyMiddleware } from 'redux'
import createSagaMiddleware from 'react-saga'
import reducer from './reducer'
import mySaga from './sagas'
const sagaMiddleware= createSagaMiddleware()
conststore = createStore(reducer, applyMiddleware(sagaMiddleware))
sagaMiddleware.run(mySaga)
```

如何判断项目需要引入 Redux？

并非所有应用程序都需要 Redux，是否引入 Redux 由以下决定

- 正在构建的应用程序类型
- 需要解决的问题类型
- 哪些工具可以更好地解决问题

Redux 可以共享和管理状态

- 通过可预测的行为来帮助回答：状态何时、何处、为什么及如何改变
- 增加概念、代码和限制，增加学习成本和项目复杂度

平衡利弊，在以下情况引入 Redux 最有用

- 应用程序许多地方都需要状态
- 应用程序的状态经常更新
- 更新状态的逻辑比较复杂
- 项目较大，需要比较多的人协作
- 想查看状态何时、何处、为什么及如何改变

如何设置引入 Redux 后的项目目录？

小型项目

- 将所有 actions 和 reducers 作为与组件、容器或页面同级的目录存放

```
- components
- pages / containers
- actions
-reducers
```

- 页面或容器 actions 和 reudcers 与其本身放到一起

```
-pages / containers
- HomePage
  - HomePage.jsx
  - HomePageAction.jsx
  -HomePageReducer.jsx
```

大型项目

提取公共组件，进一步拆分 actions 为同步 actions.js 和异步 sagas.js

- react-boilerplate 基于React的工程项目脚手架及最佳实践样板

```
- component #公共组件
-pages / containers
  - HomePage
    - component # 私有组件
    - index.jsx
    - reducer.js
    - action.js#同步 actions
- sagas.js # 异步 actions
```

- react-starter-kit 使用 React、Relay和GraphQL构建Web应用程序的前端模板

```
- .github # Github配置，包括 CI/CD工作流
- .vscode # VSCode 设置，包括代码片段、推荐的扩展等
- common # 公共(共享) React组件
- core # 核心模块和实用功能
- store
  - createStore.js # 创建 store，允许注入reducer
  - reducers.js #默认或全局的 reducers
  - states.js #初始状态
- dialogs # 实现模态对话框的React组件
- fragments # 公共(共享) 中继片段
- hooks # React 钩子，例如 useLocation(), useCurrentUser() 等
- icons # 自定义图标 React 组件
-menu # 实现弹出菜单的 React 组件
- public # 静态资源和页面，例如 robots.txt、index.html 等
- scripts # 自动化脚本，例如 yarn deploy npm
- theme # 应用程序主题、颜色、字体、内外边距等
- workers # Cloudflare Worker 或云计算、云函数、Lambda 脚本(反向代理、SSR)
- config# 每个环境的客户端应用程序设置
- index.js/ index.ts # 应用程序入口
- routes # 应用程序路由和页面(屏幕)组件
  - HomePage
    - component #私有组件
  - modules
    -reducer.js
    - action.js
  - index.jsx # 注入 modules/reducer 到 store
```

每个页面或容器组件 `index.jsx` 注入 `modules/reducer` 到 `store`

- `core/store/createStore.js`

```
import { createStore, combineReducers } from 'redux'
import reducers as initReducers from './reducers'
import states as initState from './states'
const store = createStore(initReducers, initState)
store.reducers = Object.create(null)
store.injectReducer = (key, reducer) => {
  store.reducers[key] = reducer
  store.replaceReducer(combineReducers([
    ...initReducers,
    ...store.reducers[key]
  ])))
}
export default store
```

- `routes/HomePage/index.jsx`

```
import store from '../../../core/store/createStore'
import reducer from 'modules/reducer'
store.injectReducer('homepage', reducer)
```

手写一个 `redux-thunk` ?

`redux-thunk` 允许 `dispatch` 一个函数，而不是返回 `action` 的动作创建器

- `thunk` 可用于延迟动作的调度，或仅在满足特定条件时才调度
- 内部函数接收 `dispatch` 和 `getState` 方法作为参数

源码如下

```
function createThunkMiddleware(extraArgument) {
  return ({dispatch, getState}) => next => action => {
    if (typeof action === 'function') {
      return action(dispatch, getState, extraArgument)
    }

    return next(action)
  }
}

const thunk = createThunkMiddleware()
thunk.withExtraArgument = createThunkMiddleware
export default thunk
```

Next.js

什么是 Next.js ?

Next.js 是一款用于生产环境的 React 框架

Next.js 提供生产环境所需的所有功能和最佳开发体验

- 静态及服务器端融合渲染
- 支持 TypeScript
- 智能化打包
- 路由预取

Next.js 的特点

- 零配置
 - 自动编译并打包，从一开始就位生产环境而优化
- 混合模式：SSG 和 SSR
 - 一个项目同时支持构建时预渲染页面（SSG）和请求时渲染页面（SSR），利于 SEO
- 增量静态生成
 - 在构建之后以增量的方式添加并更新静态预渲染的页面
- 支持 TypeScript
 - 自动配置并编译 TypeScript
- 快速刷新
 - 快速、可靠的实时编辑体验，已在 Facebook 级别的应用上规模上得到验证
- 基于文件系统的路由
 - 每个 `pages` 目录下的组件都是一条路由
- API 路由
 - 创建 API 端点（可选）以提供后端功能
- 内置支持 CSS
 - 使用 CSS 模块创建组件级的样式。内置对 Sass 的支持
- 代码拆分和打包
 - 采用由 Google Chrome 小组创建的，并经过优化的打包和拆分算法
- 支持环境变量、预览模式、自定义 head 标签、自动使用 polyfills

Next.js 预渲染有哪些形式？

默认情况下，Next.js 预渲染每个 页面，即为每个页面生成 HTML 文件，无需客户端 JavaScript 渲染。从而带来更好的性能和 SEO 效果

每个生成的 HTML 页面都与该页面所需的最少 JavaScript 代码关联。当浏览器加载一个页面时，其 JavaScript 代码将运行并使页面完全具有交互性

Next.js 预渲染有两种形式，主要是页面生成时机不同：

- 静态生成 (Static Generation)
 - HTML 在构建时生成，并在每次页面请求 (request) 时重用
 - 推荐使用，CDN 可以在没有额外配置的情况下缓存静态生成的页面以提高性能
- 服务器端渲染 (Server-side Rendering)
 - 在每次页面请求 (request) 时重新生成 HTML
 - 适合需要显示频繁更新的数据，页面内容会跟随每个请求而变化

为什么 Next.js 要重新设计一套路由？

Next.js 的路由基于文件系统，每个 pages 目录下的组件都是一条路由

- 首页路由
 - index.js 通过所在目录路由
 - pages/index.js → /
 - pages/blog/index.js → /blog
- 嵌套路由
 - 其它文件名的 page 自动创建路由目录
 - pages/blog/first-post.js → /blog/first-post
 - pages/dashboard/settings/username.js → /dashboard/settings/username
- 动态路由
 - [方括号包裹的文件名].js 路由路径对应方括号中的字符串将作为参数传入
 - pages/blog/[slug].js → /blog/:slug 路由路径 /blog/hello-world, slug 参数为 hello-world
- API 路由
 - pages/api 目录下任何文件都将作为 API 端点映射到 /api/*
 - 这些文件只会增加服务端文件包体积，不会增加客户端文件包的大小
- 浅路由
 - 只改变 URL
 - 不调用获取数据的方法，包括 `getServerSideProps`, `getStaticProps` 和 `getInitialProps`
 - 通过 `useRouter`, `withRouter` 获取 router
 - 解构赋值 router 得到更新后的 `pathname` 和 `query`
 - 不重新渲染页面

Next.js 的路由面向生产环境设计，简单直观，开箱即用，前后端统一管理，利于维护

Next.js 中获取数据有哪些方法？

- `getInitialProps`
 - 停止自动静态优化，Next.js 将按需每次请求渲染页面
 - 作为 Page 组件的静态方法配置，是异步函数，返回富文本对象作为 props 传入 Page
 - Next.js 9.3 及以后不建议使用，建议使用 `getStaticProps` 和 `getServerSideProps` 实现细粒度控制
- `getStaticProps`
 - Next.js 将在编译时预渲染页面
 - 生成 HTML 文件
 - 生成 JSON 文件
 - 在 Page 中 `export` 异步函数使用
 - 返回一个包含 props 键名的对象
 - 其中的 props 的键值，作为 props 传入 Page
 - 支持增量渲染：返回 `revalidate` 属性，传入数值，多少秒内重新验证，只有在数据变化时，重新渲染
 - 支持 CDN，无需额外配置
- `getStaticPaths`
 - 当页面使用动态路由时
 - `paths` 定义必须在构建时渲染为 HTML 的路径列表
 - `fallback` 支持布尔值
 - `false` 不在列表的路由，返回 404
 - `true` 不在列表中的路由，先返回如读取中的提示，同时开始预渲染
 - 在 Page 组件中，使用 `useRouter` 获得 router 对象
 - 当 `router.isFallback` 为 `true` 时，返回表示渲染中的组件或 HTML
 - 预渲染完成后，Next.js 将刚渲染的 path 加入 `paths` 路径列表
 - 通常与 `getStaticProps` 一起使用，页面路径命中 `paths` 中声明路径后，调用 `getStaticProps` 并传入动态路由参数
- `getServerSideProps`
 - 停止自动静态优化，Next.js 将在每次请求时预渲染页面
 - 在 Page 中 `export` 异步函数使用
 - 返回一个包含 props 键名的对象
 - 其中 props 的键值，作为 props 传入 Page
 - TTFB (Time to first byte) 较 `getStaticProps` 更慢
 - 支持 CDN，需要额外配置

如何在 Next.js 中为静态资源配置 CDN？

- 上传 .next/static 文件夹到 BOS 或服务器，绑定 CDN 域名，可以是主域名是子域名
- 打开 next.config.js 添加 assetPrefix 配置，注意区分生产环境

```
const isProd = process.env.NODE_ENV === 'production'
module.exports = {
  // Use the CDN in production and localhost for development
  assetPrefix: isProd ? 'https://cdn.mydomain.com' : ''
}
```

- 重新编译

AntD

你的项目为什么选择 AntD？

特性

- 提炼自企业级中后台产品的交互语言和视觉风格
 - 设计价值观：确定性、意义感、生长性和自然
- 开箱即用的高质量 React 组件
 - 通用：按钮、图标、排版
 - 布局：分割线、栅格、布局、间距
 - 导航：固钉、面包屑、下拉菜单、导航菜单、页头、分页和步骤条
 - 数据录入：自动完成、级联选择、多选框、日期选择框
 - 表单：输入框、数字输入框、提及、单选框、评分、选择器、滑动输入条、开关、时间选择框、穿梭框、树选择、上传
 - 数据展示：头像、徽标数、日历、卡片、走马灯、折叠面板、评论、描述列表、空状态、图片、列表、气泡卡片、统计数值、表格、标签页、标签、时间轴、文字提示、树形控件
 - 反馈：警告提示、抽屉、全局提示、对话框、通知提醒框、气泡确认框、进度条、结果、骨架屏，加载中
 - 其他：锚点、回到顶部、全局化配置
- 使用 TypeScript 开发，提供完整的类型定义文件
 - 可以享受组件属性输入建议
 - 定义检查的功能
- 全链路开发和设计工具体系
- 数十个国际化语言支持
 - 提供 React 组件 ConfigProvider 用于全局配置国际化文案
- 深入每个细节的主题定制能力
 - Antd 样式使用 Less 作为开发语言
 - 使用 less 提供的 modifyVars 覆盖变量
 - 支持 .umrc.ts 或 config/config.ts 的 theme 字段配置主题

- 支持 create-react-app 中定制主题

兼容环境

- 现代浏览器和 IE11
- 支持服务器渲染
- 支持 Electron
- 可选 Ant Design Mobile 支持移动端
 - 支持 Preact / React / React Native

社区支持

- Ant Design of Angular / Ant Design of Vue
- 完整文档 / 大厂背书 / 众多用户，容易找到解决方案

AntD 如何实现固定表头时，列头和内容对不齐怎么办？

- 为 Table 设置 `scroll={{ y: 任意数值 }}` 开启固定表头
- AntD 使用两个的 div 分别嵌套 table 实现固定表头效果
 - 每个 table 的 table-layout 布局算法为 fixed，即列宽度由表格宽度和列宽度设定
 - 需要手动指定每一列的宽度，以确保上下两个 table 的每一列宽度同步
 - 如果内容可能出现超长连续字段（长数字和长单词），在 columns 属性新增
 - `textWrap: 'word-break'`
 - `ellipsis: true`

AntD 如何实现虚拟列表？

- 通过 react-window 虚拟滚动方案引入 VariableSizeGrid
- 构建 renderVirtualList 函数组件，接收 `rowData, { scrollbarSize, ref, onScroll }` 作为参数
 - `rowData` 表格原始数据源，用于计算总高度、行数和显示数据
 - `scrollbarSize` 当竖直滚动条显示时，设置内容区域的宽度减去滚动条宽度
 - `ref` 用于外部直接访问和操作 VariableSizeGrid，处理水平滚动和数据重置
 - `onScroll` 当表格水平滚动时，同步虚拟列表水平滚动
- 引入 antd 的 Table 组件，设置 components 属性为 `{{ body: renderVirtualList }}`